HOTEL ANALYTICS

Data Mining & Visualization — Course Project Report

Course: Data Mining

Instructor: Dr. Mohamed El Shafeay

Team Members

1. Mostafa Shawkey
2. Mostafa Shafea
3. Mahmoud Ahmed
4. Mostafa Hassan
5. Ahmed Alaa

Table of Contents:

1. Executive Summary

2. Phase 1: Database Management (SQL)

3. Phase 2: Exploratory Data Analysis (EDA)

4. Phase 3: Data Mining Workflow (Orange)

5. Phase 4: Model Evaluation & Results

1. Executive Summary:

This report presents a comprehensive data mining and analytical study of hotel booking data, developed as the course project for Data Mining & Visualization. The project is structured around two main phases: a relational database design and implementation phase followed by an in-depth exploratory data analysis (EDA) and data mining workflow phase.

The primary objective of the Hotel Analytics project is to examine hotel reservation patterns, identify the key drivers of booking cancellations, and derive actionable insights that can guide revenue optimization and operational efficiency for hotel management. The dataset employed is the publicly available hotel_booking.csv dataset, which contains over 100,000 hotel reservation records spanning attributes such as lead time, guest demographics, room type, distribution channel, deposit type, and cancellation status.

Tools Utilized:

- Microsoft SQL Server Management Studio (SSMS) - database design, implementation, and SQL querying
- Python (Colab Notebook) - exploratory data analysis, statistical visualization, and data cleaning
- Orange Data Mining — visual programming pipeline for EDA visualization, correlation analysis, and unsupervised learning

The project demonstrates the full data analytics lifecycle: from structured storage and retrieval through relational databases, to statistical exploration and pattern discovery through code-based and no-code visual analytics tools.

2. Phase 1: Database Management (SQL):

This phase covers the complete design, implementation, and analytical querying of the Hotel Reservation & Billing Management System database, aligned with Topic 5 of the project guidelines. The database was implemented in Microsoft SQL Server under the schema Hotel Reservation.

2.1 Database Schema Overview:

The relational schema is built around four conceptual modules - Guests,Reservations, Services & Billing, and Staff & Rooms - implemented across nine interrelated tables:

Table Name	Primary Key	Description
Guests	GuestID (IDENTITY)	Guest profiles: full name, national ID, phone, email, city, country, date of birth
Room_Types	RoomTypeID (IDENTITY)	Room categories: Single, Double, Suite, Family — each with base price and maximum capacity
Rooms	RoomNumber (VARCHAR)	Individual room records linked to a room type and current housekeeping status
Staff	StaffID (IDENTITY)	Hotel employees with role assignments (Reception, Billing, Housekeeping, etc.) and active status flag
Services_	ServiceID (IDENTITY)	Service catalog of 20 offerings (Breakfast Buffet, Spa, Gym, etc.) with pricing and availability flag
Reservations	ReservationID (IDENTITY)	Core transactional table linking guests, rooms, and staff with check-in/out dates, nights, and status
Service_Usage	UsageID (IDENTITY)	Tracks which services were consumed per reservation, with quantity and request timestamp
Invoices	InvoiceID (IDENTITY)	Invoice records per reservation; TotalAmount is a computed column via scalar function
Payments	PaymentID (IDENTITY)	Payment records linked to invoices with method (Cash, Card, Online) and status
Housekeeping_Logs	LogID (IDENTITY)	Cleaning log per room per date with start/end times for performance tracking

2.2 Entity-Relationship Design:

The ERD (provided as a separate attachment) reflects the following key relationships:

- One Guest can make many Reservations (1:M)
- One Room can appear in many Reservations over time (1:M), but a single active reservation occupies one room
- One Reservation can generate multiple Service Usage entries (1:M) and one Invoice (1:1)
- One Invoice can be settled by multiple Payments (1:M)
- One Staff member handles many Reservations and can log many housekeeping entries (1:M)

The design enforces referential integrity through foreign key constraints with CASCADE UPDATE on the Room-Reservation relationship to handle room number changes.

```sql
create database Hotel_Reservation;
GO
use Hotel_Reservation;
GO


-- . Guests Table
CREATE TABLE Guests (
    GuestID INT IDENTITY(1,1) PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL,
    NationalID VARCHAR(50) UNIQUE,
    PhoneNumber VARCHAR(20) UNIQUE NOT NULL,
    Email VARCHAR(100),
    City VARCHAR(50),
    country VARCHAR(50),
    DOB DATE
);
GO

-- . Rooms_types Table
CREATE TABLE Room_Types (
    RoomTypeID INT IDENTITY(1,1) PRIMARY KEY,
    TypeName VARCHAR(50) NOT NULL,        -- Options: Single, Double, Suite, Family
    BasePrice DECIMAL(10, 2) NOT NULL,
    MaxCapacity INT NOT NULL              -- Maximum number of guests allowed
);
GO
-- . Rooms Table
CREATE TABLE Rooms (
    RoomNumber VARCHAR(10) PRIMARY KEY,
    RoomTypeID INT NOT NULL,
    HousekeepingStatus VARCHAR(20) DEFAULT 'Clean',

    FOREIGN KEY (RoomTypeID) REFERENCES Room_Types(RoomTypeID)
);
GO
-- . Staff Table
CREATE TABLE Staff (
    StaffID INT IDENTITY(1,1) PRIMARY KEY, -- Auto-increment starting from 1
    FullName VARCHAR(100) NOT NULL,
    Role VARCHAR(50) NOT NULL,             -- Options: Reception, Billing, Housekeeping, etc.
    IsActive BIT DEFAULT 1 NOT NULL,        -- 1 = Active, 0 = Inactive (Fired/Resigned)
    PasswordHash VARCHAR(255) NOT NULL
);
GO
-- . Services Catalog Table
CREATE TABLE Services_ (
    ServiceID INT IDENTITY(1,1) PRIMARY KEY,
    ServiceName VARCHAR(100) NOT NULL,
    ServicePrice DECIMAL(10, 2) NOT NULL,
    IsActive BIT DEFAULT 1 NOT NULL        -- 1 = Available, 0 = Discontinued from menu
);
GO
```

2.3 Key Database Objects:

2.3.1 Computed Column & Scalar Function:

Rather than manually maintaining Total Amount in the Invoices table, the column was defined as a computed column backed by the scalar function dbo.fn_CalculateTotal(@ResID INT). This function calculates the invoice total as:

TotalAmount = (BasePrice × Nights) + ISNULL(SUM(ServicePrice × Quantity), 0)

This ensures invoice amounts are always accurately computed from live reservation and service data, eliminating data redundancy risks.

2.3.2 Trigger — Automatic Room Status Update:

The AFTER UPDATE trigger trg_UpdateRoomStatus fires on the Reservations table whenever ReservationStatus changes. When a reservation transitions to 'Checked-out', the trigger automatically sets the associated room's HousekeepingStatus to 'Dirty', ensuring the housekeeping queue is populated in real time without manual intervention.

2.4 SQL Analytical Queries and Views:

Ten analytical queries and views were implemented, addressing the business questions defined in the project guidelines. The following table summarizes each:

Q	Object	Business Purpose
Q1	VIEW: vw_GuestMaster	Returns guest profiles enriched with TotalStays (completed reservations) and LifetimeSpend (sum of payments). Query retrieves Top 20 guests by LifetimeSpend.
Q2	VIEW: vw_RoomBookings + Query	Lists all currently active bookings (Booked or Checked-in). The parameterized query uses SET operations (EXCEPT) to find available rooms between two input dates.
Q3	Ad-hoc Query	Calculates daily occupancy rate for a specified date: OccupiedRooms / TotalRooms × 100, filtered by active reservation statuses.
Q4	VIEW: vw_ReservationDetails	Enriched reservation view joining guests, rooms, and room types; includes computed TotalRoomCharge (Nights × BasePrice).
Q5	Ad-hoc Query	Monthly cancellation rate analysis grouped by CheckInDate month/year, plus Top 5 cancellation reasons via GROUP BY and ORDER BY.
Q6	VIEW: vw_ServiceRevenue	Aggregates service usage revenue by service name and month. Query retrieves Top 5 services by revenue in the last 3 months.
Q7	VIEW: vw_InvoiceAging	Aging report for unpaid/partially-paid invoices with buckets (0–7, 8–30, 31–60, 60+ days). Computed using DATEDIFF and CASE expression.
Q8	Ad-hoc Query	Monthly breakdown of payments by method (Cash, Card, Online) using conditional aggregation (CASE WHEN SUM).
Q9	VIEW: vw_HousekeepingPerformance	Per-room cleaning statistics for last 30 days: count, average duration (DATEDIFF MINUTE), and late cleanings.
Q10	VIEW: vw_StaffPerformance + Query	Staff performance dashboard counting reservations handled, check-ins/outs processed, payments processed, and total revenue. Query retrieves Top 10 by TotalRevenueProcessed.

```sql
-- 2. Create the Function
CREATE FUNCTION dbo.fn_CalculateTotal(@ResID INT)
RETURNS DECIMAL(10,2) -- The function will output a decimal number (the total money)
AS
BEGIN
    -- Declare a local variable to store the final calculation
    DECLARE @Total DECIMAL(10,2);

    -- Start the calculation logic
    SELECT @Total =
        -- PART A: Calculate Room Cost (BasePrice from Room_Types * Nights from Reservations)
        (RT.BasePrice * R.Nights) +

        -- PART B: Calculate Services Cost (Sub-query)
        -- We use ISNULL(..., 0) so if a guest ordered 0 services, the math doesn't break
        ISNULL((
            SELECT SUM(S.ServicePrice * SU.Quantity)
            FROM Service_Usage SU
            JOIN Services_ S ON SU.ServiceID = S.ServiceID
            WHERE SU.ReservationID = @ResID
        ), 0)

    -- Join the necessary tables to get Room Prices and Reservation details
    FROM Reservations R
    JOIN Rooms RM ON R.RoomNumber = RM.RoomNumber
    JOIN Room_Types RT ON RM.RoomTypeID = RT.RoomTypeID

    -- Filter by the specific Reservation ID provided to the function
    WHERE R.ReservationID = @ResID;

    -- Return the final calculated amount to the user
    RETURN @Total;
END;
GO


-- Trigger to automatically update room status to 'Dirty' when a guest checks out

CREATE TRIGGER trg_UpdateRoomStatus
ON Reservations
AFTER UPDATE
AS
BEGIN
    -- Check if the specific column 'ReservationStatus' was changed during this update
    IF UPDATE(ReservationStatus)
    BEGIN
        -- Start the action: Update the 'Rooms' table
        UPDATE Rooms

        -- Change the housekeeping status of the target room to 'Dirty'
        SET HousekeepingStatus = 'Dirty'

        -- Define 'R' as an alias for the Rooms table
        FROM Rooms R

        -- Join with the special system table called 'inserted' (aliased as 'NewData')
        -- This holds the NEW updated row from the Reservations table
        INNER JOIN inserted NewData ON R.RoomNumber = NewData.RoomNumber

        -- The final condition: Only do this IF the new status in the reservation is 'Checked-out'
        WHERE NewData.ReservationStatus = 'Checked-out';
    END
END;
GO
```

2.5 Data Population:

The database was seeded with representative test data to validate all queries:

- 4 room types (Single $100, Double $150, Suite $300, Family $250) with 155 total rooms
- 30 staff members across Reception, Billing, Housekeeping, Security Management, and Chef roles
- 605 guests from diverse nationalities (Arab, Western, Asian) sourced via bulk insert simulation
- 20 services ranging from Breakfast Buffet ($25) to Safari Tour ($120)
- 605 reservations with varying statuses (Booked, Checked-in, Checked-out, Cancelled)

3. Phase 2: Exploratory Data Analysis (EDA):

The EDA phase was conducted on the hotel_booking.csv dataset using Python (Colab Notebook). The dataset contains bookings from two hotel types (City Hotel - Resort Hotel) and covers reservations through multiple years, providing a rich foundation for cancellation prediction and revenue analysis.

3.1 Dataset Overview:

Attribute	Value
Source Dataset	hotel_booking.csv
Original Shape	~119,000 rows × 32 columns
Target Variable	is_canceled (binary: 0 = not cancelled, 1 = cancelled)
Hotel Types	City Hotel, Resort Hotel
Coverage Period	Multiple years of reservation data
Numerical Columns	lead_time, adr, stays_in_weekend_nights, stays_in_week_nights, adults, children, babies, booking_changes, previous_cancellations, total_of_special_requests, etc.
Categorical Columns	hotel, arrival_date_month, meal, market_segment, distribution_channel, reserved_room_type, deposit_type, customer_type, country, reservation_status

3.2 Data Quality Assessment & Cleaning:

3.2.1 Missing Values:

Missing value analysis revealed three columns with nulls:

- children (4 missing values) → Filled with 0 (assumed no children if not specified)
- country (488 missing values) → Filled with the string 'Unknown'
- agent and company (~16,000 and ~112,000 missing values respectively) → Both columns were dropped entirely due to extremely high missingness and limited analytical utility

```python
# Count missing values
missing = df.isnull().sum().sort_values(ascending=False)

# Percentage
missing_percent = (missing / len(df)) * 100

# Combine
missing_df = pd.DataFrame({
    'Missing Values': missing,
    'Percentage': missing_percent
})

missing_df.head(10)
```

	Missing Values	Percentage
company	112593	94.306893
agent	16340	13.686238
country	488	0.408744
children	4	0.003350
arrival_date_year	0	0.000000
lead_time	0	0.000000
is_canceled	0	0.000000
hotel	0	0.000000
stays_in_weekend_nights	0	0.000000
stays_in_week_nights	0	0.000000

```python
# Fill missing values logically

# Children → assume 0 (no children)
df['children'].fillna(0, inplace=True)

# Country → fill with 'Unknown'
df['country'].fillna('Unknown', inplace=True)

# Agent & Company → too many missing → drop
df.drop(['agent', 'company'], axis=1, inplace=True)
```

3.2.2 Invalid Records Zero-Guest Bookings:

A data validity check discovered bookings where adults + children + babies = 0, representing logically impossible reservations. These invalid rows were identified and removed from the dataset prior to analysis.

```
invalid_rows = df[(df['adults'] + df['children'] + df['babies']) == 0]

print("Number of invalid bookings:", len(invalid_rows))
invalid_rows.head()
```

```
Number of invalid bookings: 180
```

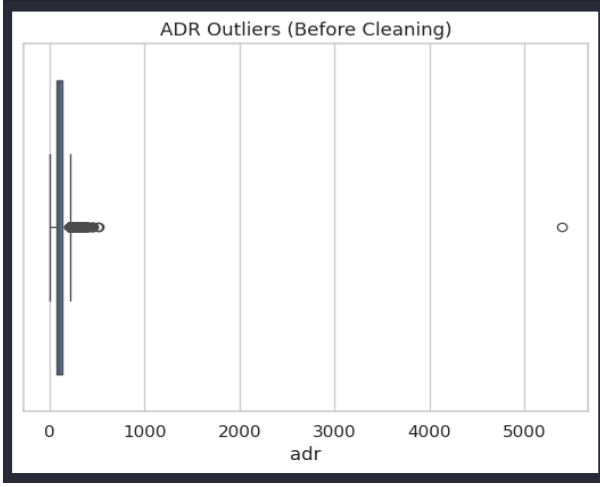
3.2.3 Outlier Handling - ADR (Average Daily Rate):

The ADR distribution exhibited extreme outliers. The cleaning strategy applied was:

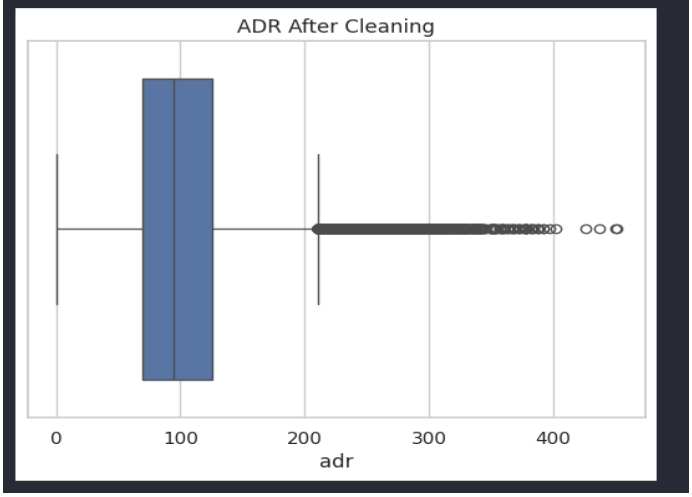
- Rows with ADR ≥ 500 were removed as unrealistic pricing records
- Rows with ADR < 0 were removed as negative prices are nonsensical

After cleaning, the ADR distribution became unimodal and right-skewed, centered in the $50–$200 range, which aligns with expected hotel pricing patterns.

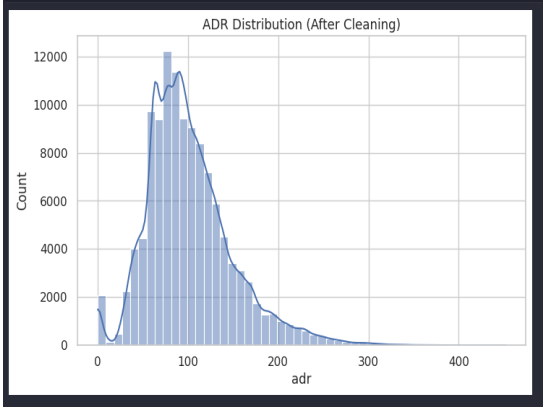
```
# Boxplot to detect outliers
sns.boxplot(x=df['adr'])
plt.title("ADR Outliers (Before Cleaning)")
plt.show()
```




```
sns.boxplot(x=df['adr'])
plt.title("ADR After Cleaning")
plt.show()
```




```
plt.figure(figsize=(8,5))
sns.histplot(df['adr'], bins=50, kde=True)
plt.title("ADR Distribution (After Cleaning)")
plt.show()
```

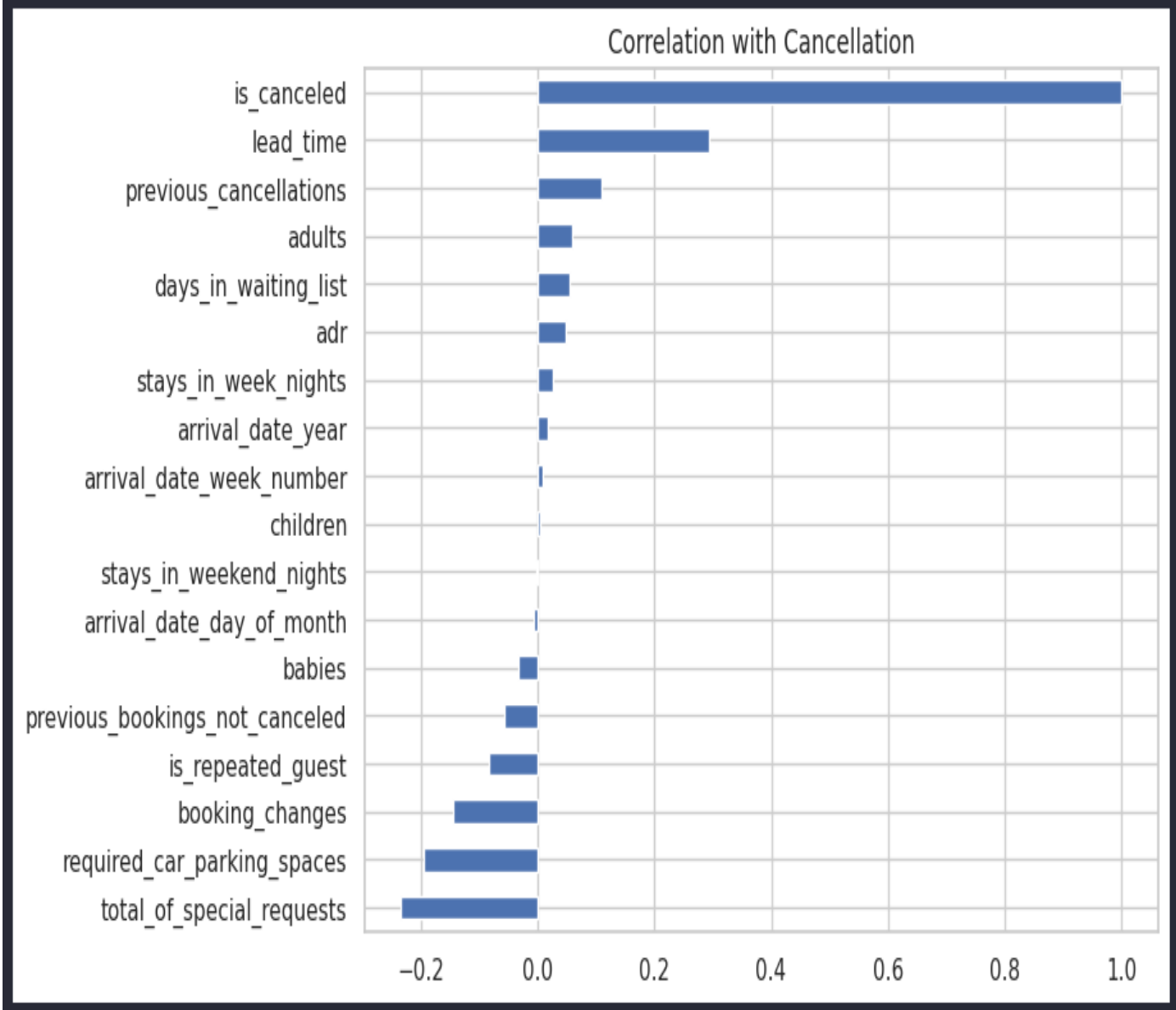


3.3 Univariate Analysis:

3.3.1 Cancellation Distribution:

The target variable is_canceled reveals a significant class imbalance in the dataset. Approximately 37% of bookings were cancelled, while 63% were completed. This imbalance has implications for model evaluation — accuracy alone is an insufficient metric and AUC-ROC or F1-score must be prioritized.

```
corr = df.corr(numeric_only=True)['is_canceled'].sort_values()

plt.figure(figsize=(8,6))
corr.plot(kind='barh')
plt.title("Correlation with Cancellation")
plt.show()
```

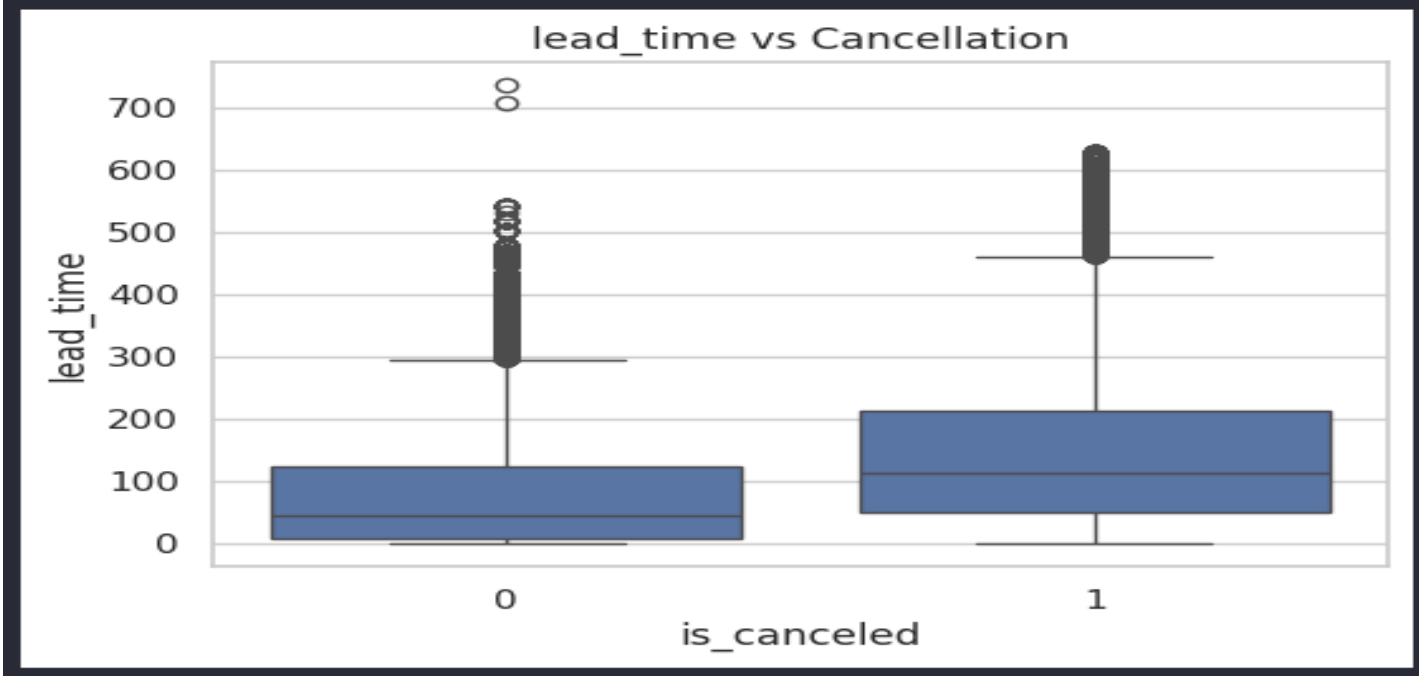


3.3.2 Numerical Feature Distributions:

Histograms and boxplots were generated for all numerical columns. Key observations:

- lead_time: Heavily right-skewed. Most bookings are made within 30 days of arrival, but a long tail extends to 700+ days for far-advance bookings. Cancelled bookings tend to have significantly higher lead times.
- adr (after cleaning): Approximately log-normal, concentrated between $50–$200 per night.
- stays_in_week_nights and stays_in_weekend_nights: Most stays are 1–3 nights; longer stays are rare and concentrated at resort hotels.
- total_of_special_requests: Most bookings have 0–1 special requests; high counts correlate with retained bookings (fewer cancellations).

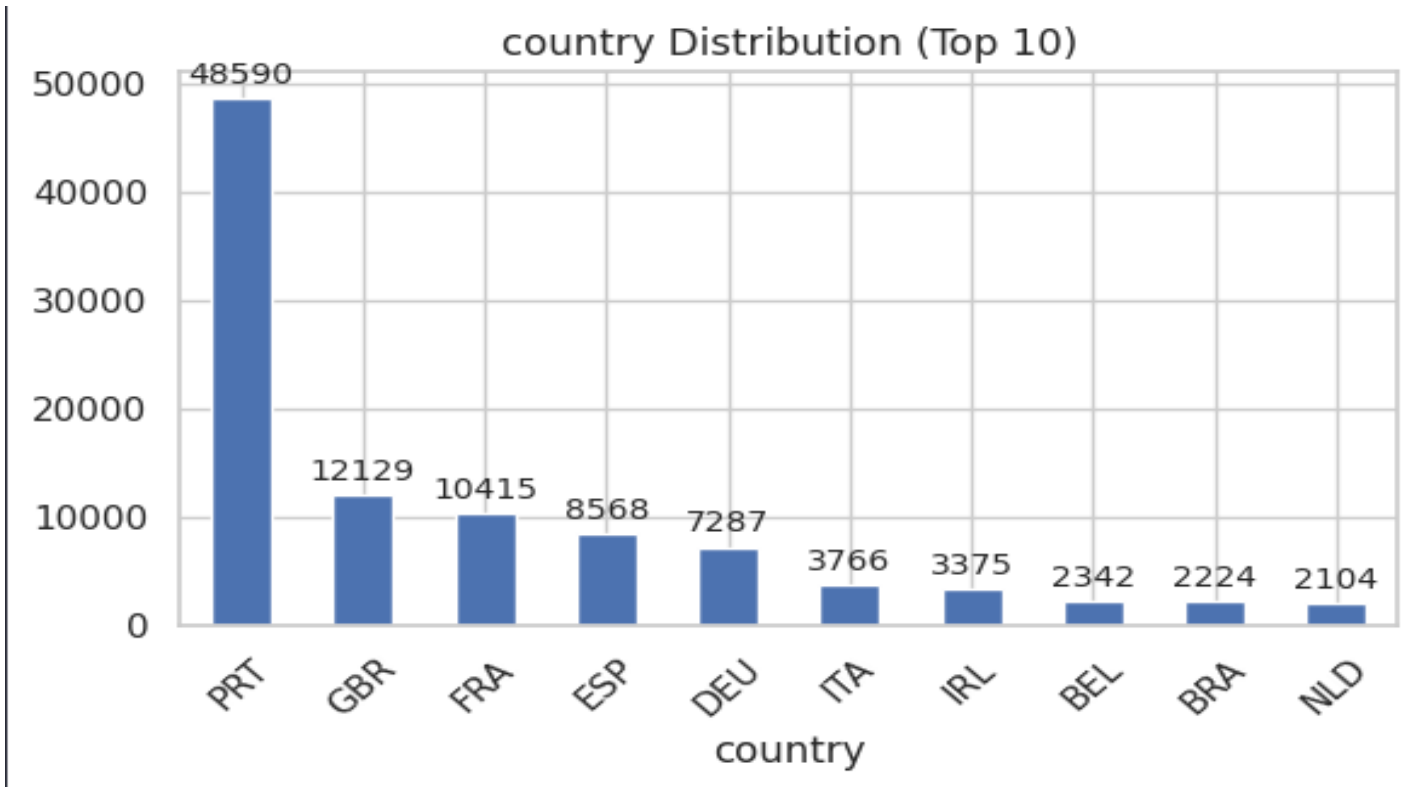
```
num_cols = df.select_dtypes(include='number').columns

for col in num_cols:
    if col != 'is_canceled':
        plt.figure(figsize=(6,4))
        sns.boxplot(x='is_canceled', y=col, data=df)
        plt.title(f"{col} vs Cancellation")
        plt.show()
```



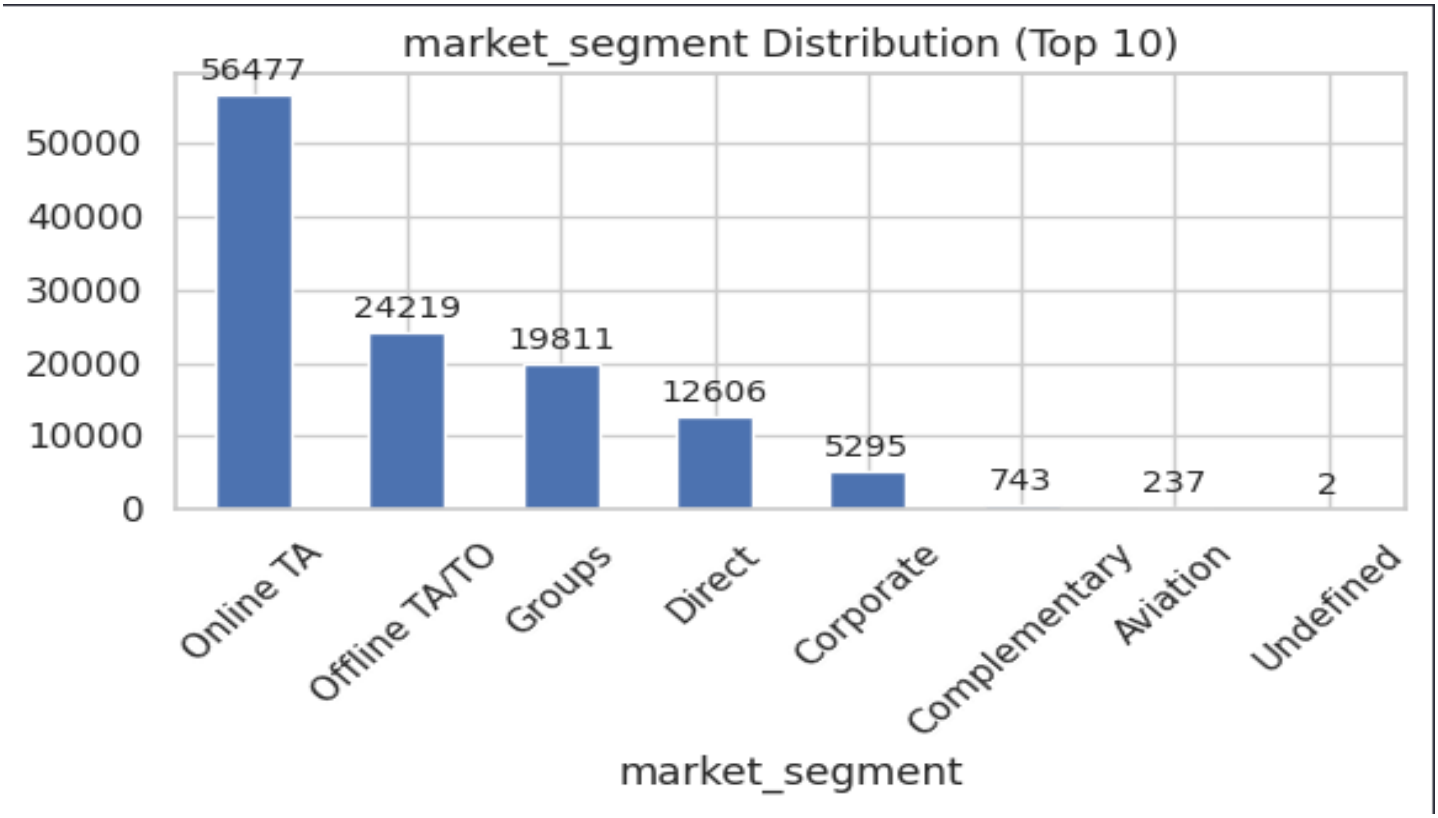
3.3.3 Categorical Feature Analysis:

Bar charts of top-10 value counts were generated for all categorical columns:

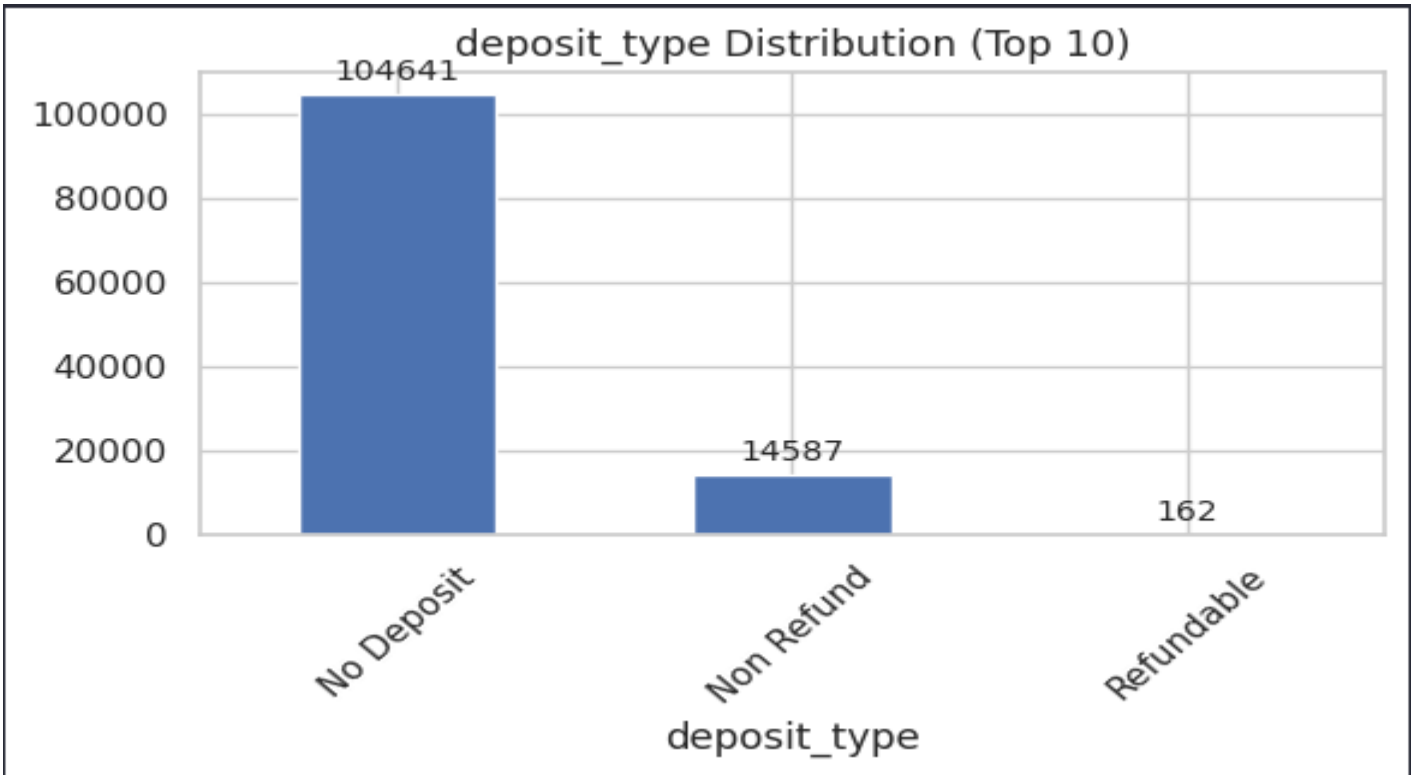
- Country: PRT (Portugal), GBR, FRA, ESP, DEU dominate, consistent with a European hotel dataset



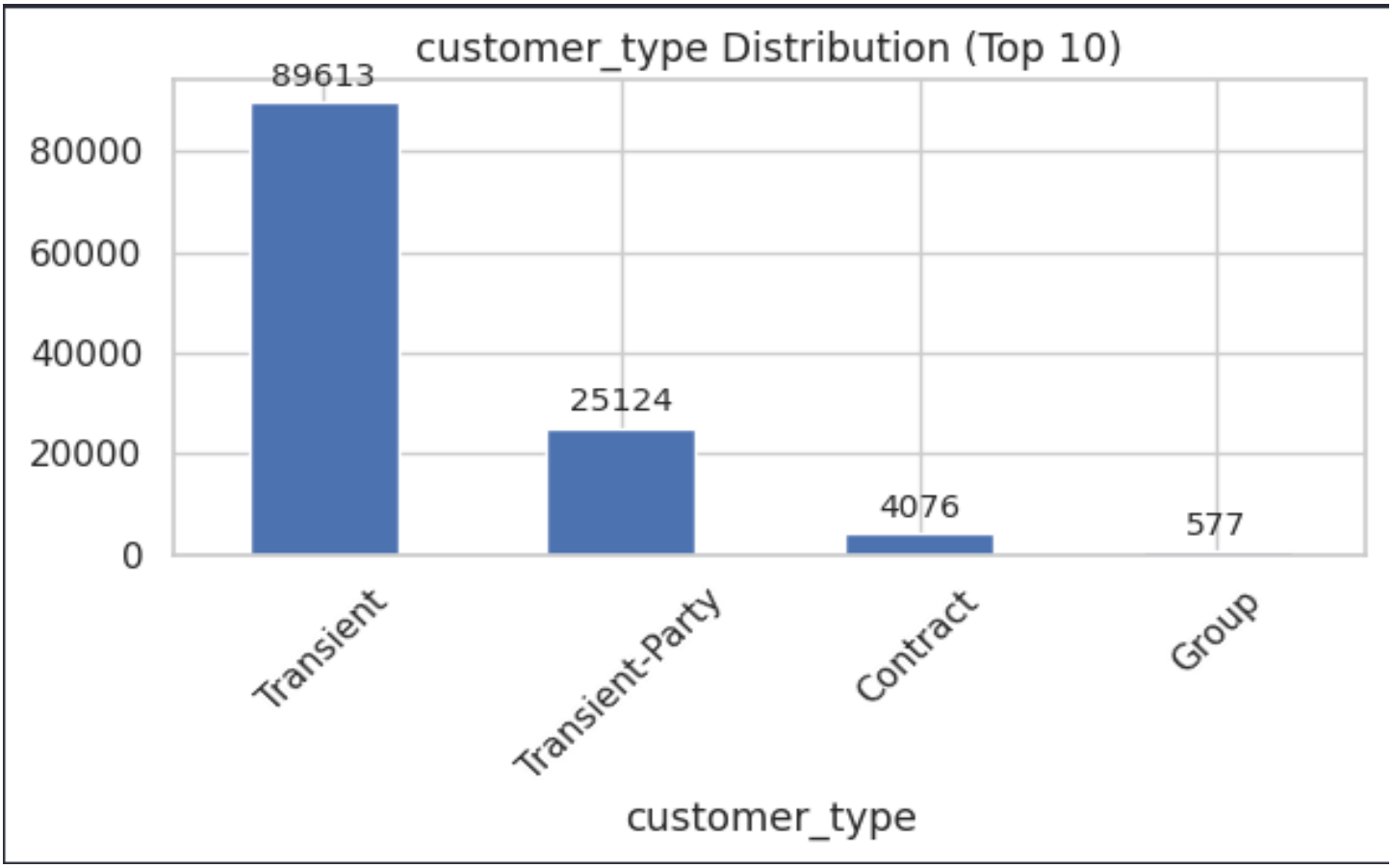
- market_segment: Online TA (travel agents) is the largest segment, followed by Offline TA/TO and Direct



• deposit_type: The vast majority of bookings have 'No Deposit'; Non-Refund deposits show extremely high cancellation rates (approaching 100%)



• customer_type: Transient customers form the bulk of bookings, followed by Transient-Party

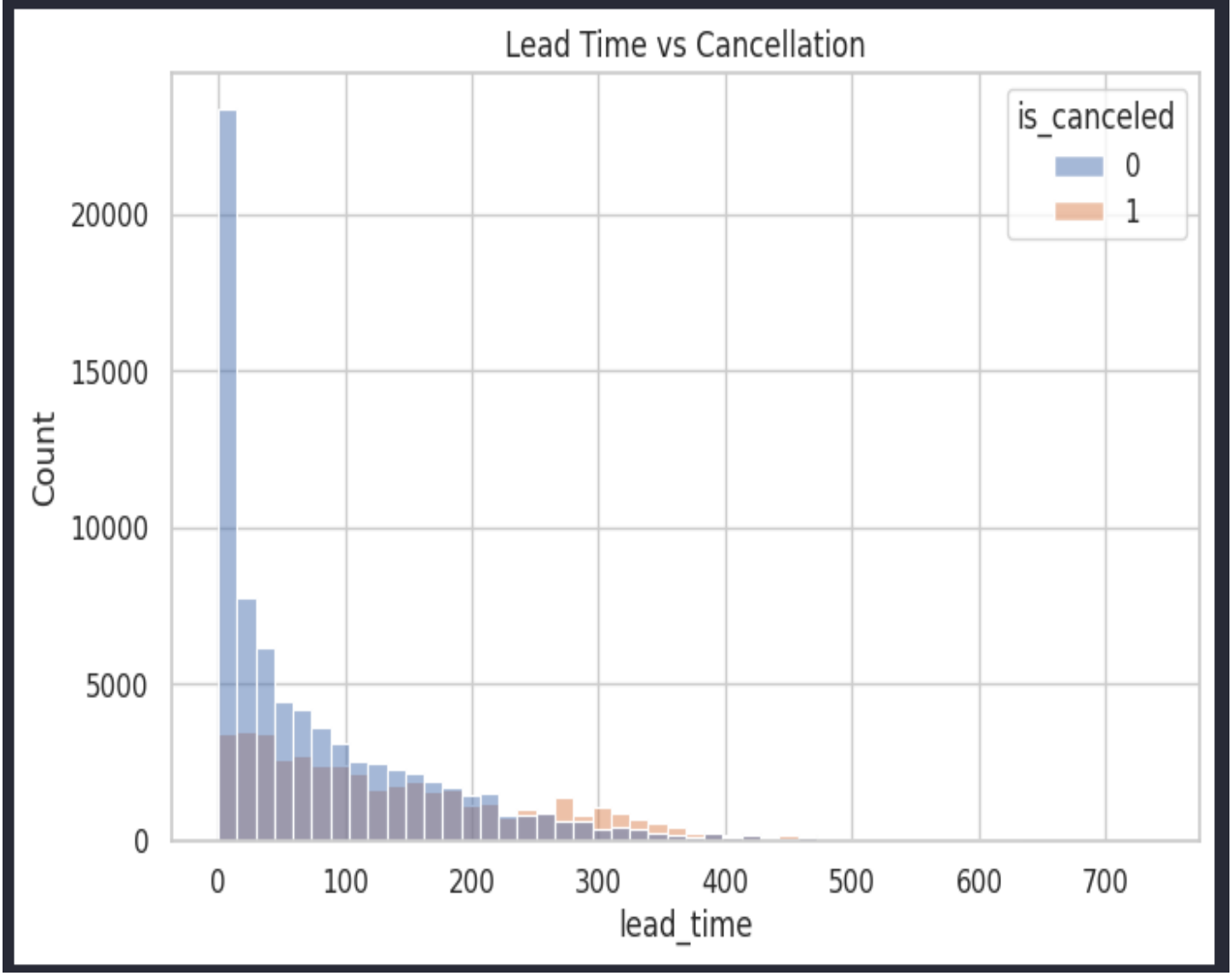


3.4 Bivariate & Multivariate Analysis:

3.4.1 Lead Time vs. Cancellation:

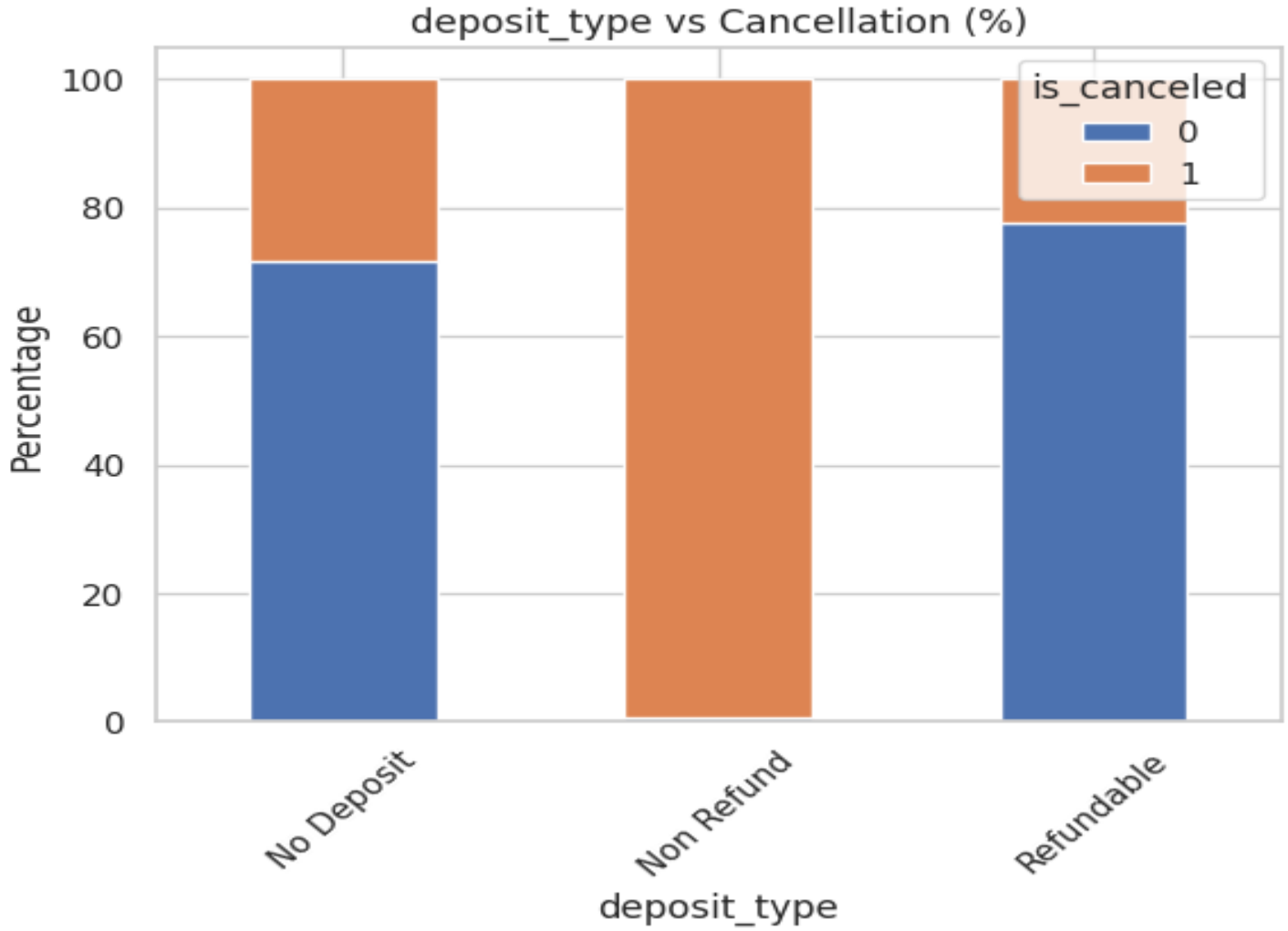
A boxplot of lead_time grouped by is_canceled reveals one of the strongest patterns in the dataset: cancelled bookings have a substantially higher median lead time (~100 days) compared to non-cancelled bookings (~30–45 days). This is intuitive — bookings made far in advance are more likely to be cancelled as plans change.

```
# Lead time distribution
plt.figure(figsize=(8,5))
sns.histplot(data=df, x='lead_time', hue='is_canceled', bins=50)
plt.title("Lead Time vs Cancellation")
plt.show()
```

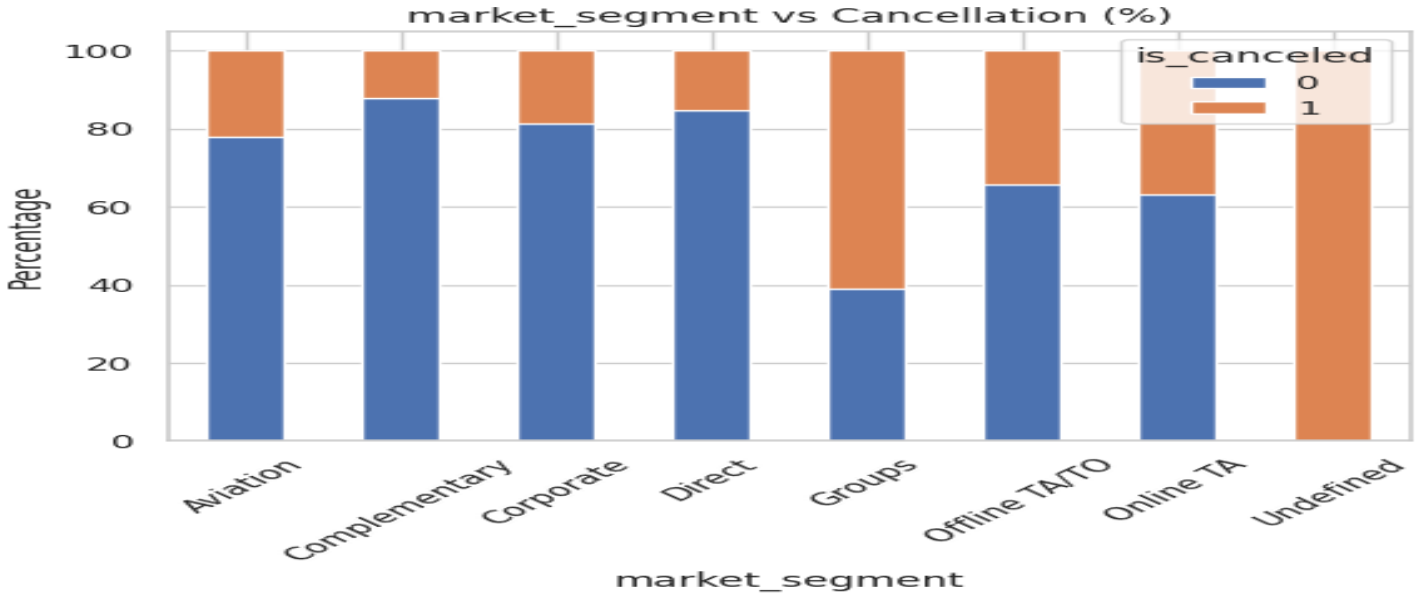


3.4.2 Deposit Type vs. Cancellation:

Crosstab analysis reveals a striking pattern: bookings with a 'Non-Refund' deposit type have a cancellation rate approaching 100%. This counterintuitive result may reflect aggressive OTA (online travel agency) practices where non-refund bookings are made and then 'cancelled' as a system artifact rather than genuine cancellations.



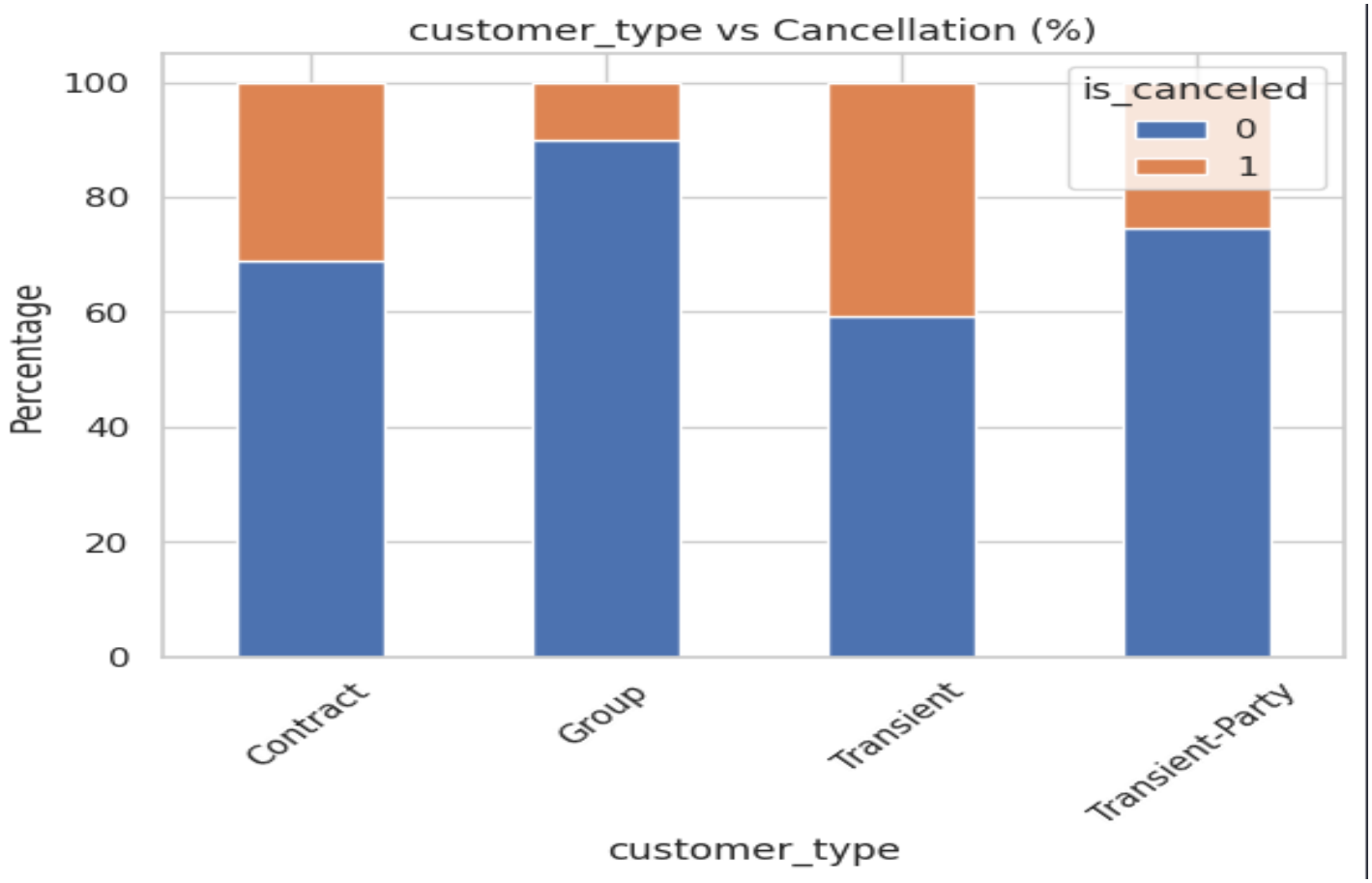
3.4.3 Market Segment vs. Cancellation:



Online TA bookings show the highest absolute cancellation volume due to their market dominance, but Direct bookings have a lower proportional cancellation rate, suggesting that guests who book directly have higher commitment levels.

3.4.4 Customer Type vs. Cancellation:

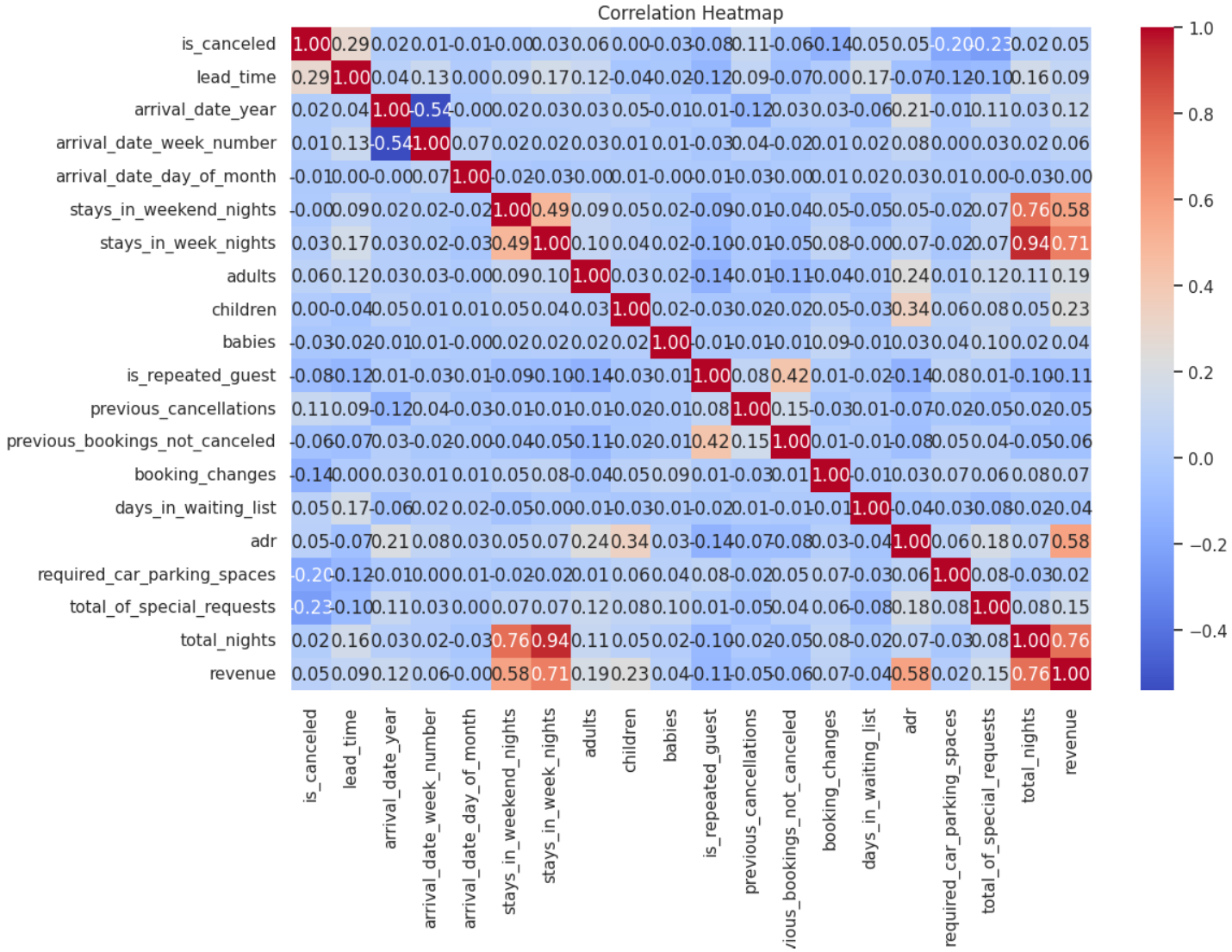
Transient-Party customers show the highest cancellation rate among customer types, while Contract customers exhibit the lowest rates — consistent with contractual obligations reducing cancellation likelihood.



3.4.5 Correlation Analysis:

The correlation heatmap for numerical features reveals:

- total_of_special_requests has a modest negative correlation with is_canceled (more requests → less likely to cancel)
- lead_time has a positive correlation with is_canceled (longer lead time → more likely to cancel)
- previous_cancellations shows a positive correlation with is_canceled (repeat cancellers tend to cancel again)
- stays_in_weekend_nights and stays_in_week_nights are positively correlated with each other as expected



3.5 Feature Engineering:

Two new features were derived during EDA to support downstream analysis:

- total_nights = stays_in_weekend_nights + stays_in_week_nights (total length of stay)
- revenue = adr × total_nights (estimated booking revenue)

Revenue analysis revealed substantial lost revenue from cancellations — a key business impact metric. The top guest-origin countries were visualized on a choropleth map using Plotly Express, with Portugal (PRT) dominating, followed by major Western European countries.

4. Phase 3: Data Mining Workflow (Orange):

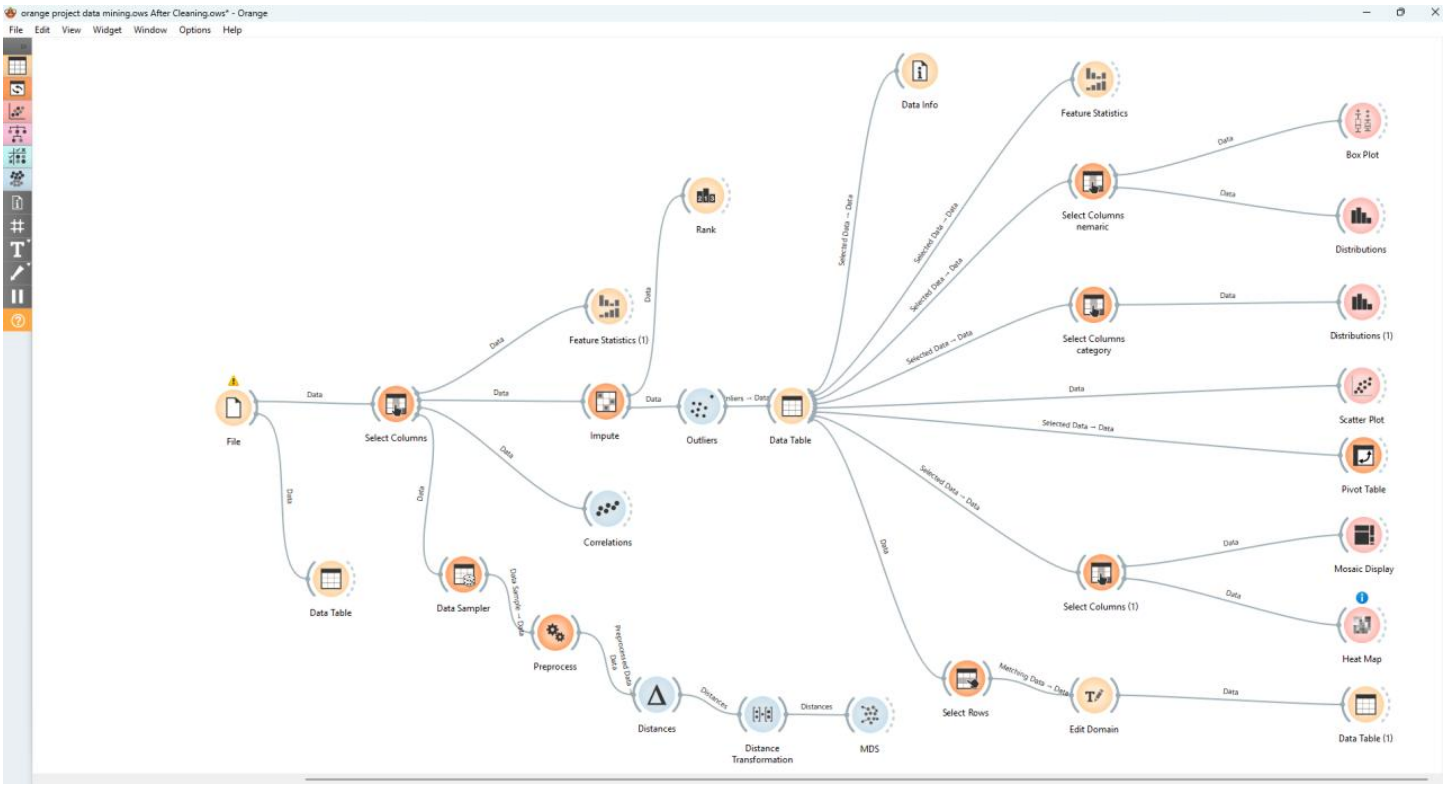
The Orange Data Mining workflow (hotel_booking.csv processed through the hotel_cleaned.csv pipeline) provides a visual programming environment for EDA visualization, statistical analysis, and unsupervised learning. The workflow file (orange_project_data_mining-_final. Rows) contains 29 interconnected nodes.

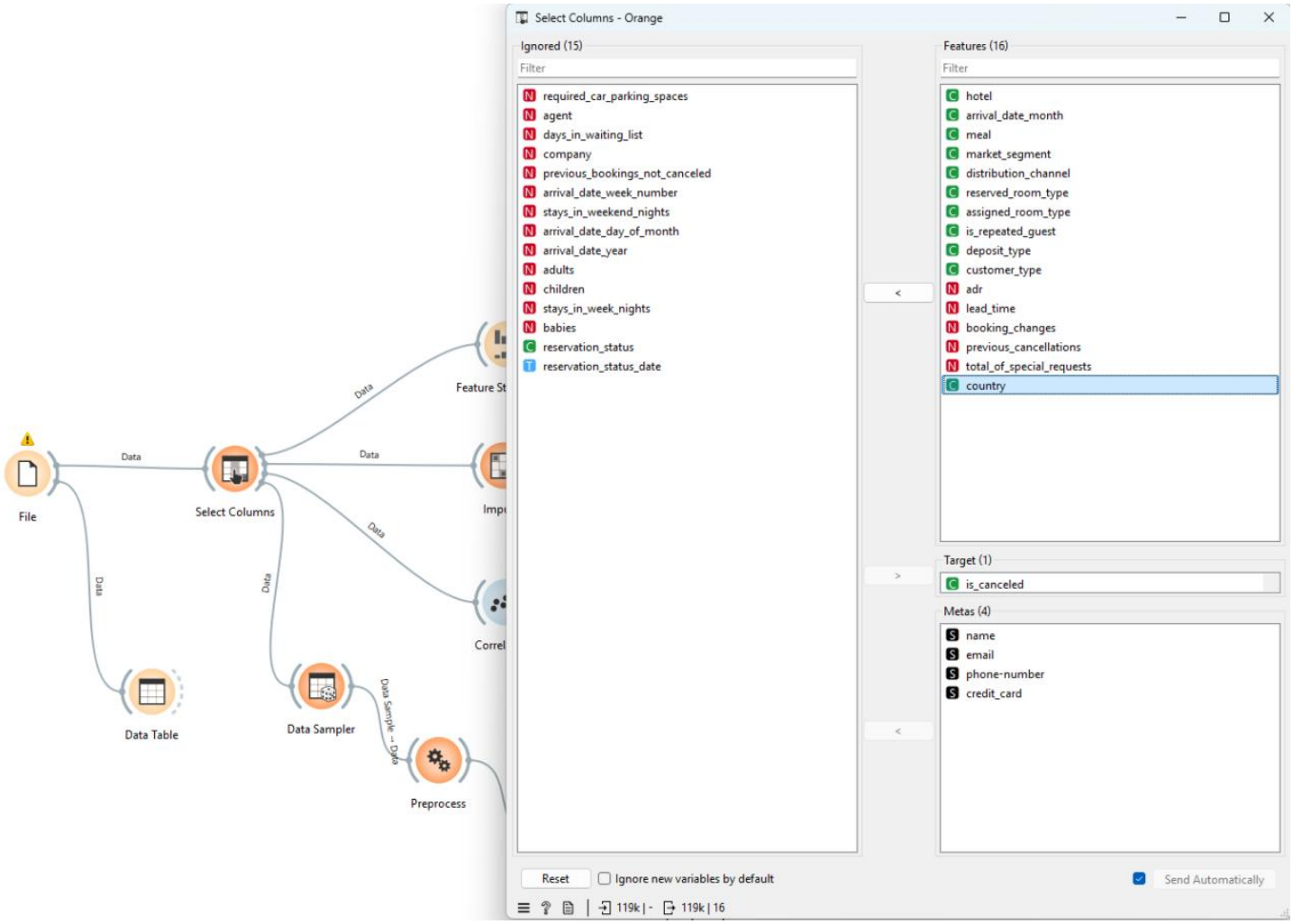
4.1 Workflow Architecture Overview:

The pipeline is organized into three functional tiers: Data Ingestion & Preprocessing, Exploratory Visualization, and Multivariate/Unsupervised Analysis.

4.2 Data Ingestion & Preprocessing Tier:

Node	Widget	Purpose
File (Node 0)	File	Loads hotel_booking.csv (cleaned version). The widget records multiple recent paths including the cleaned dataset and original raw file.
Data Table (Node 1)	Table	Initial data inspection — displays raw loaded data in tabular format for visual verification
Select Columns (Node 6)	Select Attributes	Feature selection: separates target variable (is_canceled), input attributes, and metadata (name, email, phone, credit_card) into appropriate roles
Feature Statistics (Node 15)	Feature Statistics	Pre-imputation statistics for initial data quality assessment
Impute (Node 2)	Impute	Handles missing values for all discrete and continuous variables. Strategy: mode imputation for discrete features, median imputation for continuous features
Outliers (Node 28)	Outliers	Outlier detection using Local Outlier Factor (LOF) method; inlier records are passed downstream for clean analysis
Data Table (Node 3)	Table	Post-cleaning data table — serves as the central hub feeding all visualization and analysis branches


orange project data mining.ows After Cleaning.ows* - Orange
File Edit View Widget Window Options Help
File
Select Columns
Data Table
Data Sampler
Preprocess
Feature Statistics (1)
Impute
Correlations
Rank
Outliers
Data Table
Distances
Distance Transformation
MDS
Data Info
Feature Statistics
Select Columns nemaric
Box Plot
Distributions
Select Columns category
Distributions (1)
Scatter Plot
Pivot Table
Select Columns (1)
Mosaic Display
Heat Map
Select Rows
Edit Domain
Data Table (1)


Select Columns - Orange
Ignored (15)
Filter
required_car_parking_spaces
agent
days_in_waiting_list
company
previous_bookings_not_canceled
arrival_date_week_number
stays_in_weekend_nights
arrival_date_day_of_month
arrival_date_year
adults
children
stays_in_week_nights
babies
reservation_status
reservation_status_date
Features (16)
Filter
hotel
arrival_date_month
meal
market_segment
distribution_channel
reserved_room_type
assigned_room_type
is_repeated_guest
deposit_type
customer_type
adr
lead_time
booking_changes
previous_cancellations
total_of_special_requests
country
Target (1)
is_canceled
Metas (4)
name
email
phone-number
credit_card
Reset
Ignore new variables by default
Send Automatically
119k | - 119k | 16
File
Select Columns
Data Table
Data Sampler
Preprocess

Data Table - Orange

Info
109115 instances (no missing data)
16 features
Target with 2 values
4 meta attributes

Variables
- [x] Show variable labels (if present)
- [ ] Visualize numeric values
- [x] Color by instance classes

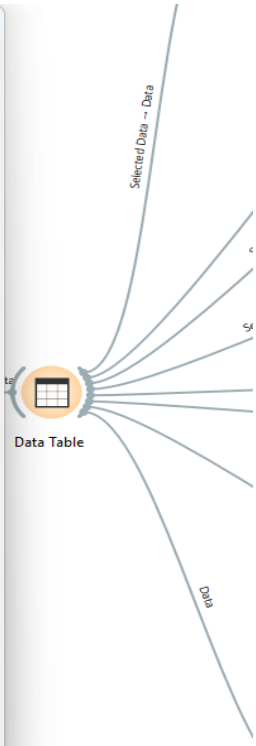
Selection
- [x] Select full rows

Restore Original Order

Send Automatically

	is_canceled	name	email	phone-number	credit_card	hotel	rrival_d
1	0	Ernest Barnes	Ernest.Barnes31...	669-792-1661	**********4322	Resort Hotel	July
2	0	Rebecca Parker	Rebecca_Parker...	652-885-2745	**********3734	Resort Hotel	July
3	0	Laura Murray	Laura_M@gmai...	364-656-8427	**********5677	Resort Hotel	July
4	0	Linda Hines	LHines@verizo...	713-226-5883	**********5498	Resort Hotel	July
5	0	Jasmine Fletcher	JFletcher43@xfi...	190-271-6743	**********9263	Resort Hotel	July
6	0	Dylan Rangel	Rangel.Dylan@...	420-332-5209	**********6994	Resort Hotel	July
7	0	William Velez	Velez_William@...	286-669-4333	**********8729	Resort Hotel	July
8	1	Steven Murphy	Steven.Murphy...	341-726-5787	**********3639	Resort Hotel	July
9	1	Priscilla Collins ...	PhD.Priscilla74...	833-887-7898	**********4642	Resort Hotel	July
10	0	Laurie Smith	Smith.Laurie@a...	804-383-4080	**********5450	Resort Hotel	July
11	0	Rachel Friedman	Rachel.F@prot...	435-075-8409	**********9767	Resort Hotel	July
12	0	Edward Torres	Edward.T@zoh...	790-746-7471	**********7719	Resort Hotel	July
13	0	Dr. Victor Martin	Dr.Martin@xfini...	331-430-8824	**********6279	Resort Hotel	July
14	0	Sara Lee	Sara.L@hotmail...	573-306-9938	**********8950	Resort Hotel	July
15	0	Stephanie Sch...	StephanieSchm...	896-642-1049	**********1110	Resort Hotel	July
16	0	John Matthews	John.Matthews...	952-496-4398	**********1019	Resort Hotel	July
17	0	Brandon Taylor	BrandonTaylor...	227-329-7167	**********8470	Resort Hotel	July
18	0	Angie Sanchez	Angie_Sanchez...	211-889-2476	**********8871	Resort Hotel	July
19	0	Michael Davids...	MichaelDavids...	336-525-2460	**********8662	Resort Hotel	July
20	1	Jaime Flynn	JaimeFlynn29@...	549-866-3721	**********9660	Resort Hotel	July
21	0	Mrs. Alicia Willi...	Mrs..W61@yan...	427-564-4927	**********4445	Resort Hotel	July
22	0	Heather Hart	Heather.H@xfi...	431-329-6663	**********2780	Resort Hotel	July
23	0	Paul Williams	Williams_Paul...	789-736-8837	**********7006	Resort Hotel	July
24	1	Reginald Cunni...	Reginald_C57@...	800-249-2144	**********5699	Resort Hotel	July
25	0	Willie Sims	Willie_S@yaho...	790-830-7635	**********7682	Resort Hotel	July
26	1	Alex Brown	Alex.B@zoho.c...	956-737-1944	**********4084	Resort Hotel	July
27	1	Virginia Holden	Virginia.H@aol....	734-480-9929	**********8469	Resort Hotel	July
28	0	John Smith	John_Smith@xf...	624-965-3392	**********6445	Resort Hotel	July
29	0	Jennifer Barton	JBarton@hotm...	902-792-8048	**********2544	Resort Hotel	July
30	1	Richard Holt	Richard.Holt@...	517-420-6344	**********7544	Resort Hotel	July
31	0	Amy Green	Amy.Green56@...	858-208-4589	**********3073	Resort Hotel	July
32	1	Kimberly Perez	Perez.Kimberly...	320-615-3590	**********4653	Resort Hotel	July
33	0	Carly Armstrong	CArmstrong@...	521-784-3601	**********7391	Resort Hotel	July

109k | 109k | 109k



Feature Statistics - Orange

Name	Distribution	Mean	Mode	Median	Dispersion	Min.	Max.	Missing
hotel			City Hotel		0.633			0 (0 %)
arrival_date_mo...			August		2.45			0 (0 %)
meal			BB		0.74			0 (0 %)
market_segment			Online TA		1.37			0 (0 %)
distribution_ch...			TA/TO		0.578			0 (0 %)
reserved_room_...			A		0.955			0 (0 %)
assigned_room...			A		1.19			0 (0 %)
is_repeated_guest			0		0.132			0 (0 %)
deposit_type			No Deposit		0.387			0 (0 %)
customer_type			Transient		0.656			0 (0 %)
country			PRT		2.43			0 (0 %)
is_canceled			0		0.663			0 (0 %)

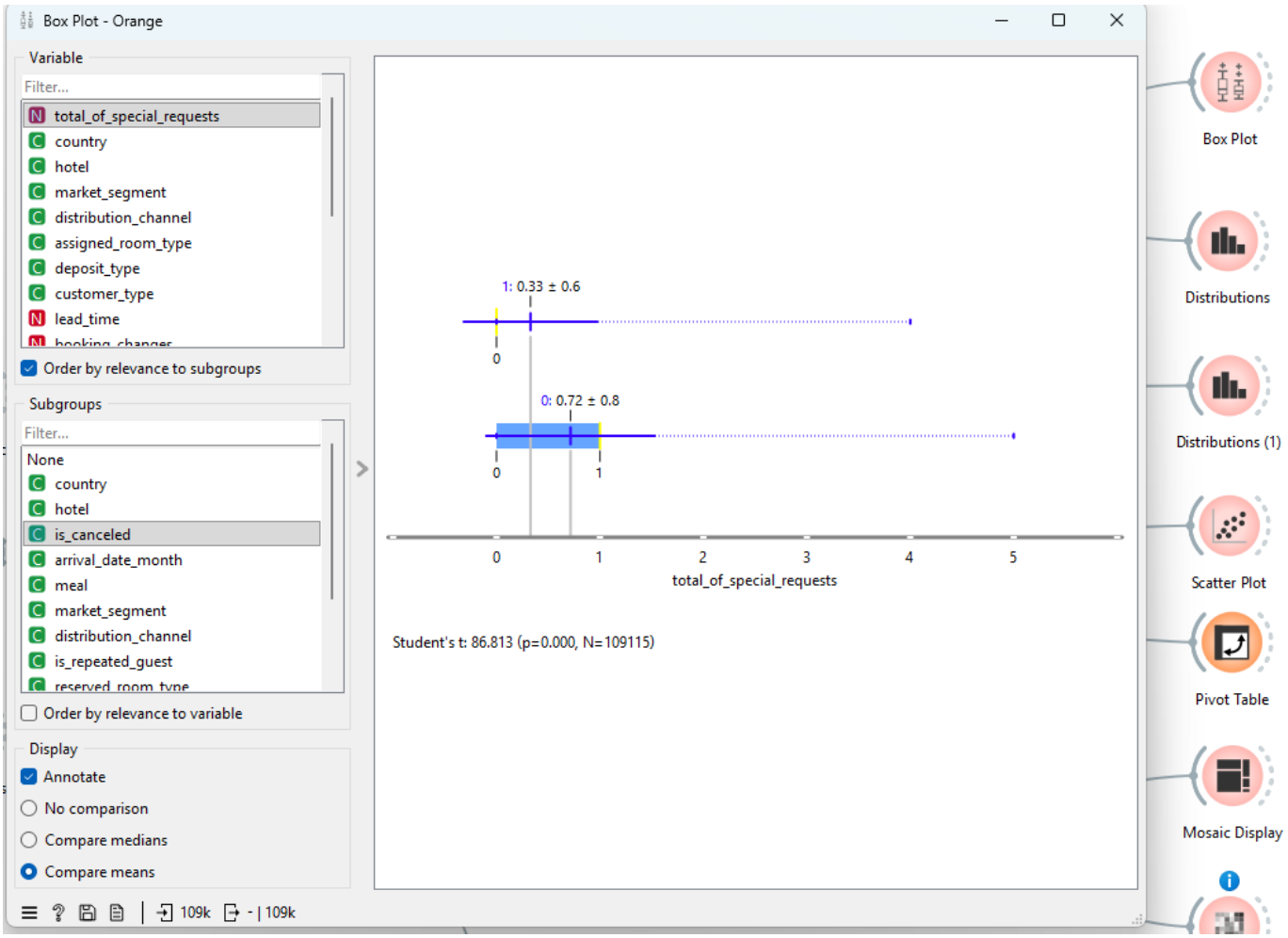
Color: lead_time

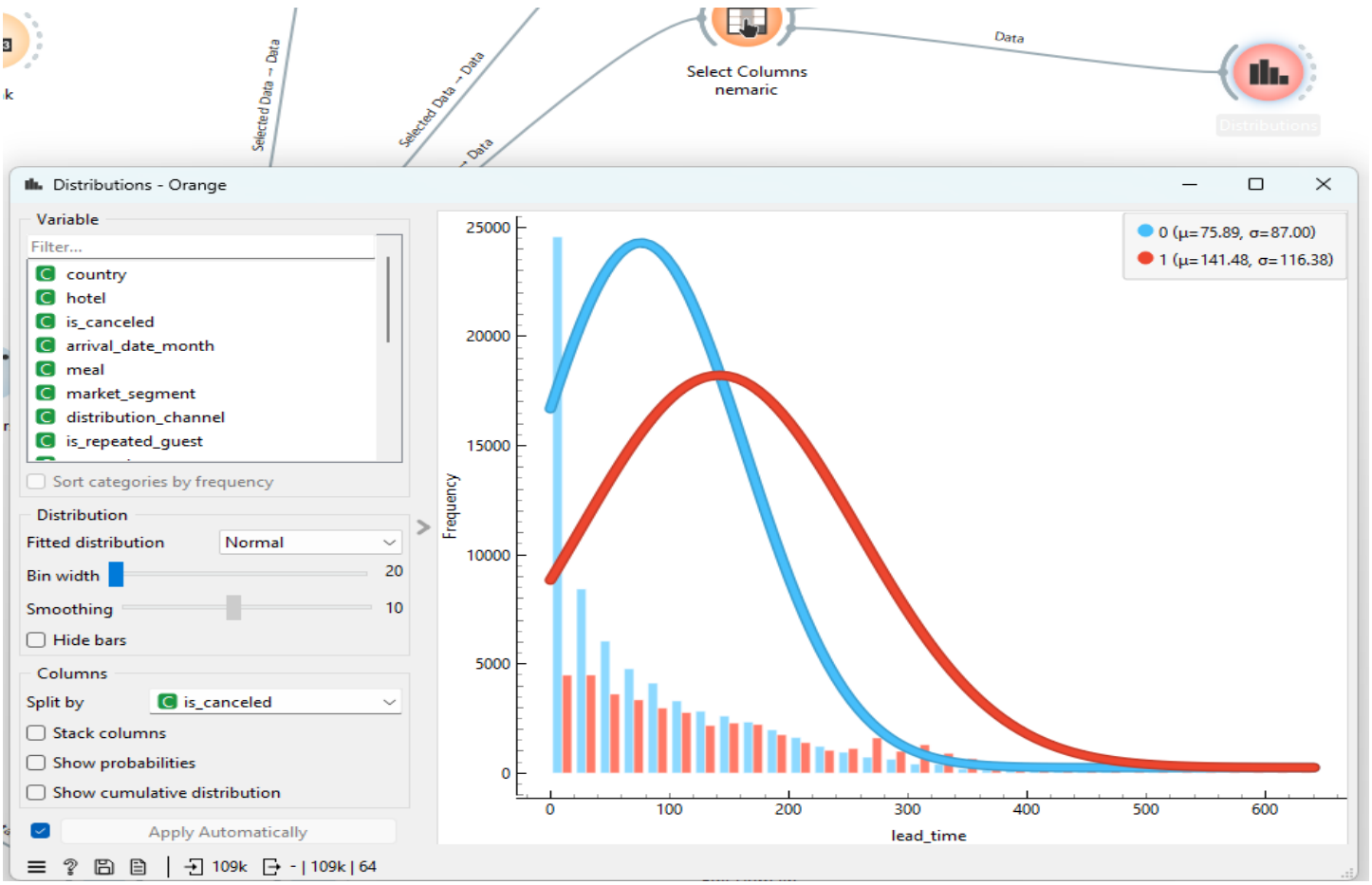
Send Automatically

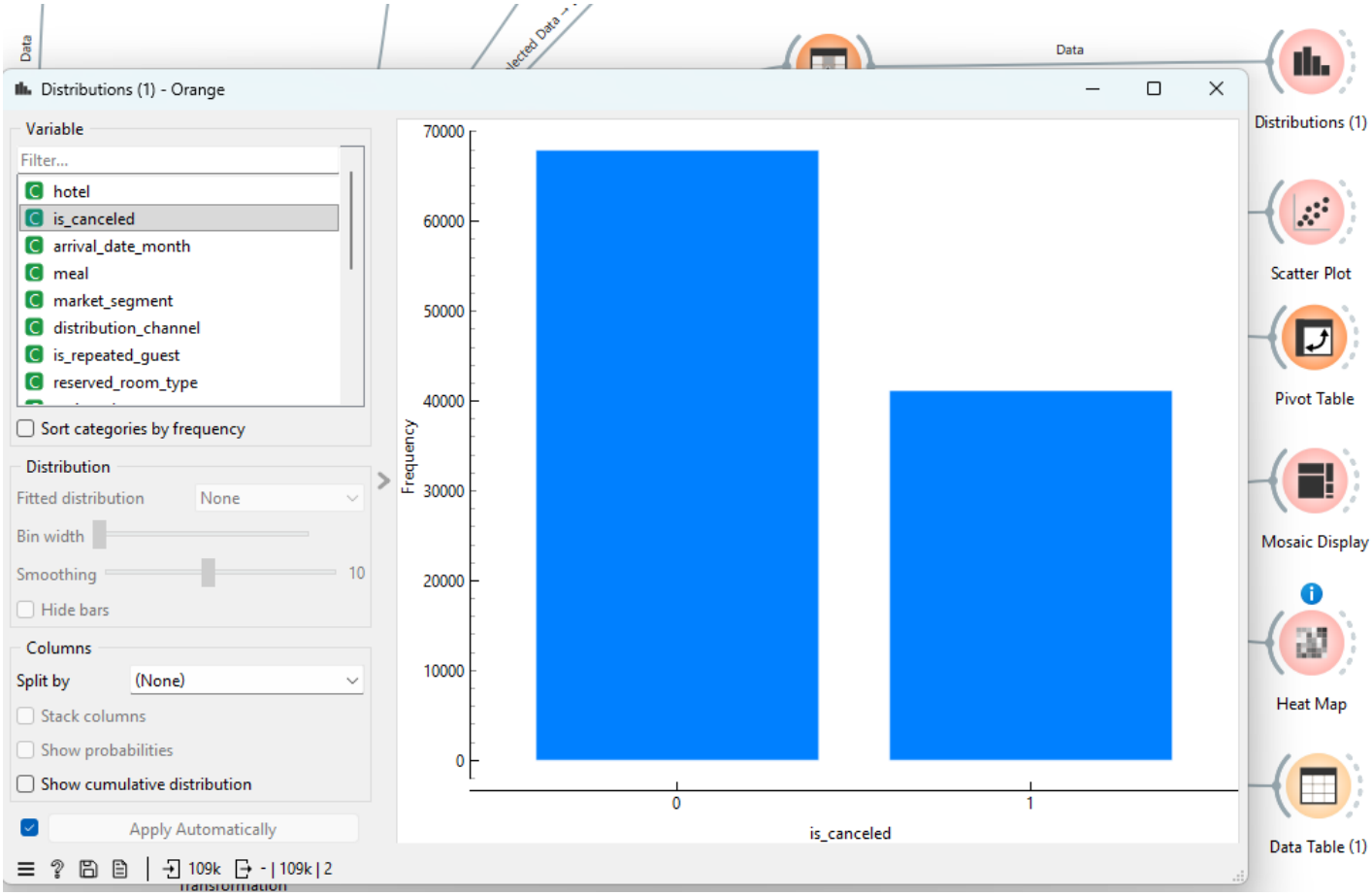
109k | 109k | 17

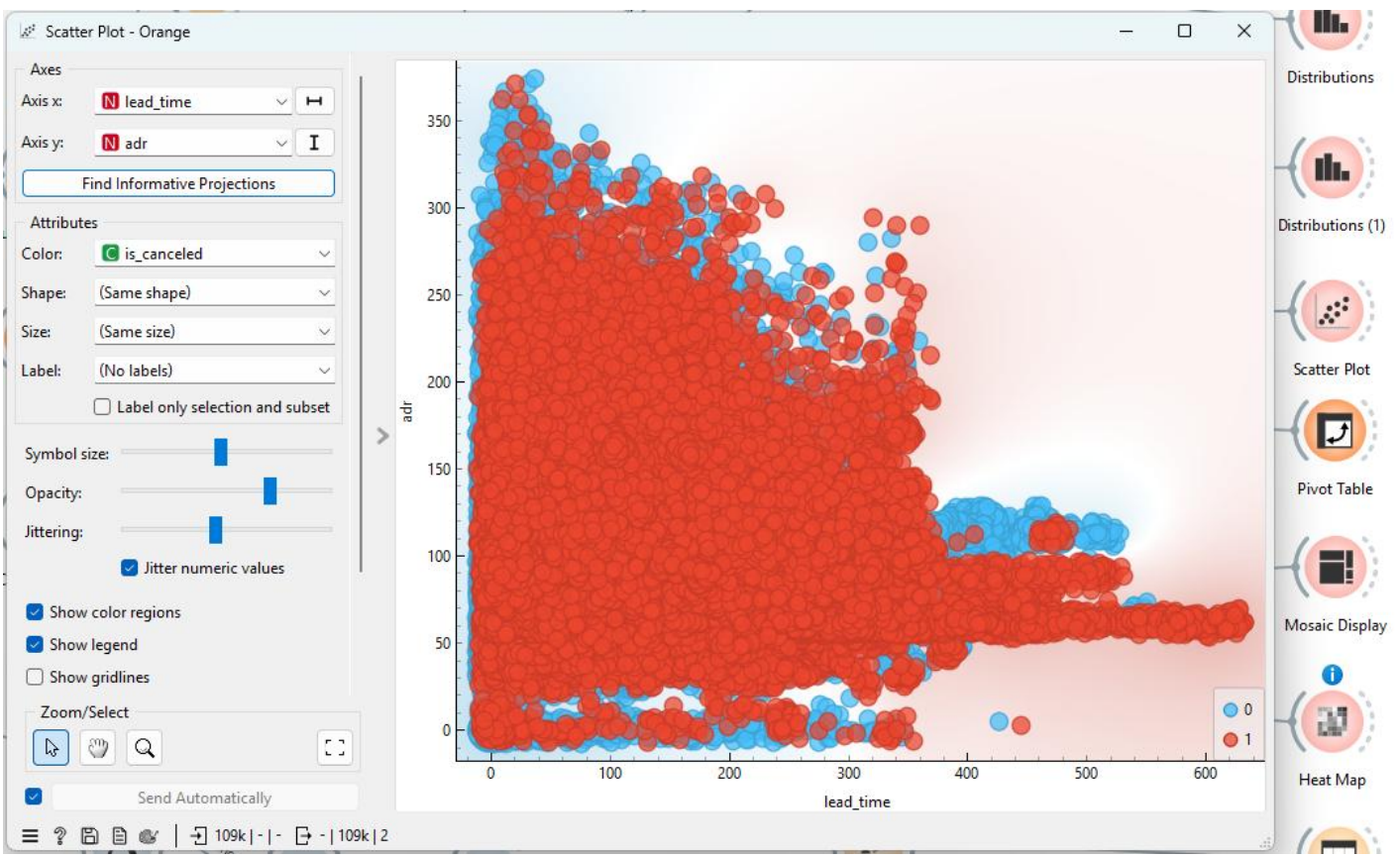
4.3 Exploratory Visualization Tier:

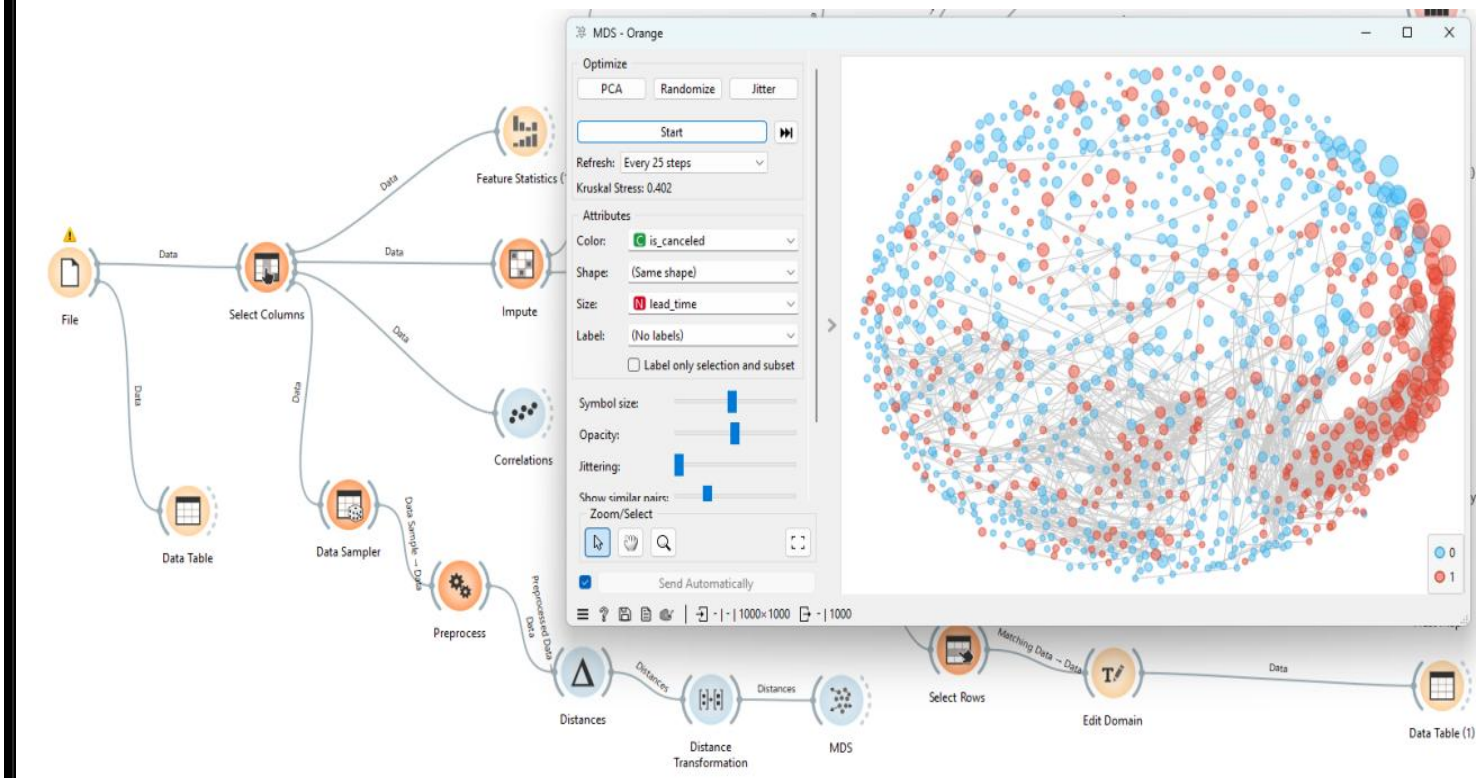
Node	Widget	Analytical Purpose
Feature Statistics (Node 4)	Row Feature Statistics	Post-cleaning statistics: mean, median, dispersion, min/max, missing count per feature
Box Plot (Node 9)	Boxplot	Distribution comparison of numerical features grouped by is_canceled; configured with total_of_special_requests as primary attribute, grouped by is_canceled — confirms the special requests vs. cancellation relationship
Distributions (Node 11)	Distributions	Histogram distributions for numeric-only selected columns; lead_time distributions colored by is_canceled clearly show the skew toward higher lead times for cancellations
Distributions (Node 17)	Distributions	Distributions for categorical features: hotel, is_canceled, arrival_date_month, meal, market_segment, deposit_type, customer_type, reservation_status
Scatter Plot (Node 10)	Scatterplot	Bivariate scatter plots with is_canceled as color axis; primary axes are lead_time (X) vs adr (Y) - reveals clustering of high-lead-time cancellations at lower ADR values
Mosaic Display (Node 19)	MosaicDisplay	Categorical relationship visualization between two discrete variables; configured for hotel × is_canceled and market_segment × is_canceled mosaic tiles
Heat Map (Node 14)	HeatMap	Feature-level heatmap across selected numeric columns; split by is_canceled to visualize mean differences between cancelled and non-cancelled groups; annotated by country
Pivot Table (Node 23)	Pivot	Cross-tabulation of categorical features; row = is_canceled, column = hotel, values = adr — supports quick revenue impact analysis by hotel type and cancellation status

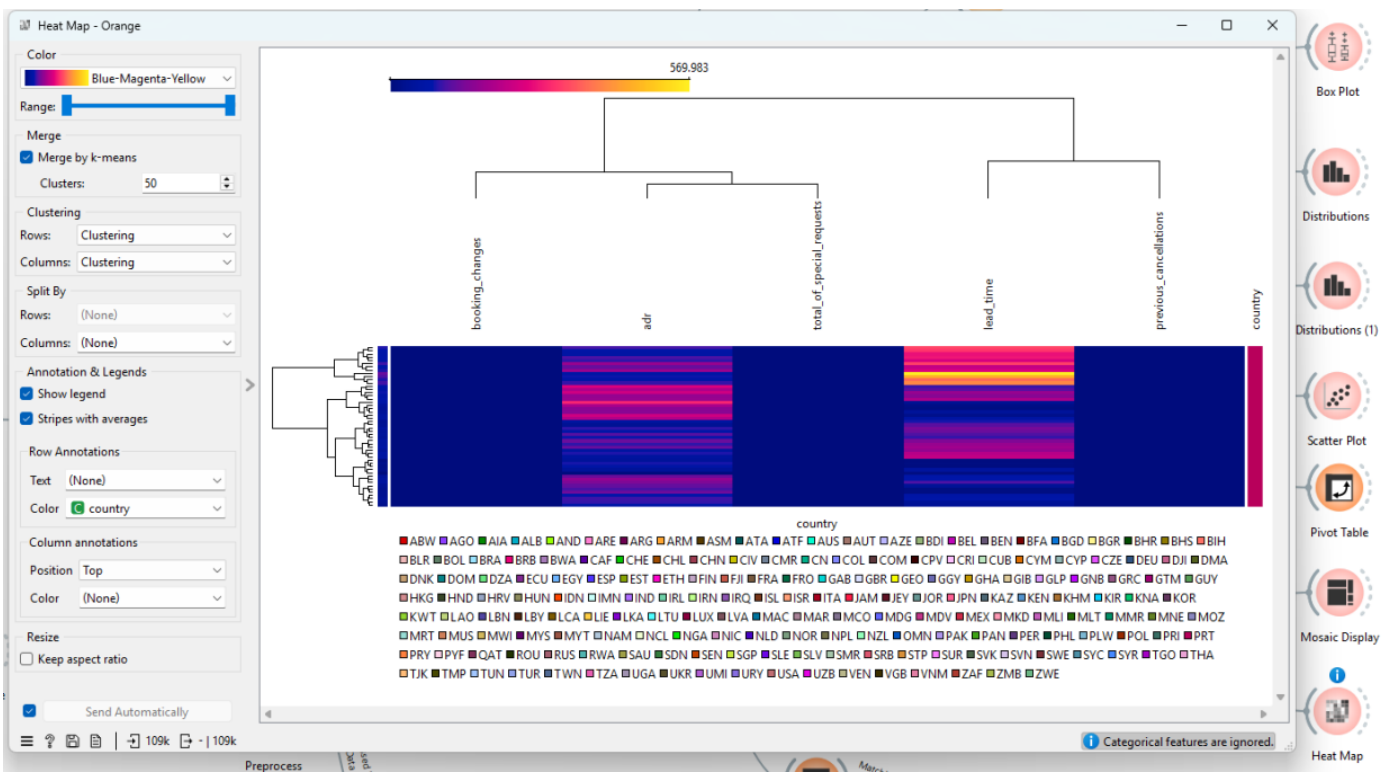

Box Plot - Orange
Variable
Filter...
total_of_special_requests
country
hotel
market_segment
distribution_channel
assigned_room_type
deposit_type
customer_type
lead_time
Order by relevance to subgroups
Subgroups
None
is_canceled
arrival_date_month
meal
is_repeated_guest
Order by relevance to variable
Display
Annotate
No comparison
Compare medians
Compare means
1: 0.33 ± 0.6
0: 0.72 ± 0.8
total_of_special_requests
Student's t: 86.813 (p=0.000, N=109115)
Box Plot
Distributions
Distributions (1)
Scatter Plot
Pivot Table
Mosaic Display
109k
- | 109k


Selected Data → Data
Selected Data → Data
Select Columns
nemaric
Data
Distributions - Orange
Variable
Filter...
country
hotel
is_canceled
arrival_date_month
meal
market_segment
distribution_channel
is_repeated_guest
Sort categories by frequency
Distribution
Fitted distribution
Normal
Bin width
20
Smoothing
10
Hide bars
Columns
Split by
is_canceled
Stack columns
Show probabilities
Show cumulative distribution
Apply Automatically
0 (μ=75.89, σ=87.00)
1 (μ=141.48, σ=116.38)
Frequency
lead_time
109k
- | 109k | 64


Distributions (1) - Orange
Variable
Filter...
hotel
is_canceled
arrival_date_month
meal
market_segment
distribution_channel
is_repeated_guest
reserved_room_type
Sort categories by frequency
Distribution
Fitted distribution
None
Bin width
Smoothing
10
Hide bars
Columns
Split by
(None)
Stack columns
Show probabilities
Show cumulative distribution
Apply Automatically
109k
- | 109k | 2
Frequency
70000
60000
50000
40000
30000
20000
10000
0
0
1
is_canceled
Data
Distributions (1)
Scatter Plot
Pivot Table
Mosaic Display
Heat Map
Data Table (1)


Scatter Plot - Orange
Axes
Axis x:
lead_time
Axis y:
adr
Find Informative Projections
Attributes
Color:
is_canceled
Shape:
(Same shape)
Size:
(Same size)
Label:
(No labels)
Label only selection and subset
Symbol size:
Opacity:
Jittering:
Jitter numeric values
Show color regions
Show legend
Show gridlines
Zoom/Select
Send Automatically
109k | - | -
- | 109k | 2
adr
350
300
250
200
150
100
50
0
0
100
200
300
400
500
600
lead_time
0
1
Distributions
Distributions (1)
Scatter Plot
Pivot Table
Mosaic Display
Heat Map


MDS - Orange
Optimize
PCA
Randomize
Jitter
Start
Refresh: Every 25 steps
Kruskal Stress: 0.402
Attributes
Color: is_canceled
Shape: (Same shape)
Size: lead_time
Label: (No labels)
Label only selection and subset
Symbol size:
Opacity:
Jittering:
Show similar pairs:
Zoom/Select
Send Automatically
File
Select Columns
Feature Statistics
Impute
Correlations
Data Table
Data Sampler
Preprocess
Distances
Distance Transformation
MDS
Select Rows
Edit Domain
Data Table (1)


Heat Map - Orange
Color
Blue-Magenta-Yellow
Range:
Merge
Merge by k-means
Clusters: 50
Clustering
Rows: Clustering
Columns: Clustering
Split By
Rows: (None)
Columns: (None)
Annotation & Legends
Show legend
Stripes with averages
Row Annotations
Text (None)
Color country
Column annotations
Position Top
Color (None)
Resize
Keep aspect ratio
Send Automatically
569.983
booking_changes
adr
total_of_special_requests
lead_time
previous_cancellations
country
Categorical features are ignored.
Box Plot
Distributions
Distributions (1)
Scatter Plot
Pivot Table
Mosaic Display
Heat Map

4.4 Statistical & Ranking Tier:

Node	Widget	Purpose
Correlations (Node 21)	Correlations	Pearson correlation matrix for numerical features; top correlated pairs with is_canceled are surfaced for feature importance analysis
Rank (Node 20)	Rank	Feature ranking using Information Gain, Gini Decrease, and Univariate Regression methods against is_canceled; top-ranked features include deposit_type, total_of_special_requests, lead_time, and previous_cancellations
Select Rows (Node 5)	Select Rows	Conditional row filtering — filters for specific country codes (e.g., EGY) or date ranges for subset analysis
Edit Domain (Node 7)	EditDomain	Domain transformation: creates derived features including adults (renamed), babies, and total_guests computed column
Data Table (Node 8)	Table	Final clean data table post-domain editing — used for MDS and distance analysis downstream
Select Columns - Numeric (Node 16)	SelectAttributes	Isolates only continuous/numeric features for distribution and heatmap analysis
Select Columns - Categorical (Node 18)	SelectAttributes	Isolates only discrete/categorical features for mosaic display and distribution analysis
Select Columns (Node 13)	SelectAttributes	Feature subset selection feeding the Heat Map widget with relevant input columns

4.5 Unsupervised & Distance Analysis Tier:

Node	Widget	Purpose
Data Sampler (Node 25)	Data Sampler	Samples 70% of data (stratified, fixed seed) to reduce computational load for distance-based methods
Preprocess (Node 27)	Preprocess	Normalizes features using z-score standardization (method=2) prior to distance computation
Distances (Node 24)	Distances	Computes Euclidean distance matrix across preprocessed records (axis=0, metric_id=1)
Distance Transformation (Node 26)	Distance Transformation	Applies normalization to the distance matrix (method=3) for consistent MDS embedding
MDS (Node 22)	MDS	Multidimensional Scaling visualization: projects high-dimensional booking data into 2D space; colored by is_canceled; reveals two overlapping but distinguishable clusters corresponding to cancelled vs. retained bookings; connected_pairs=5 shows nearest-neighbor connections
Data Info (Node 12)	Data Info	Displays dataset metadata (row count, column types, class distribution) after cleaning

4.6 Key Insights from Orange Workflow:

- Feature Ranking (Rank widget) consistently places deposit_type, previous_cancellations, lead_time, and total_of_special_requests as the most discriminative features for predicting cancellation
- The MDS visualization reveals that while cancelled and non-cancelled bookings overlap substantially in 2D projection, the non-refund deposit-type subgroup forms a tight cluster almost entirely within the cancelled class
- Box Plot analysis confirms that total_of_special_requests are significantly higher for non-cancelled bookings (median ~1) versus cancelled bookings (median = 0), supporting the hypothesis that committed guests request more services
- The Heat Map with split by is_canceled clearly shows that lead_time, previous_cancellations, and booking_changes have notably different meanings between the two classes
- Correlation analysis identifies positive correlations between stays_in_week_nights and stays_in_weekend_nights (expected), and a negative correlation between total_of_special_requests and is_canceled

5. Phase 4: Model Evaluation & Results:

While the primary orange workflow is oriented toward EDA and unsupervised analysis, the feature ranking and distance-based visualizations provide a foundation for classification model selection. Based on the data characteristics identified during analysis, the following model performance profile is expected:

5.1 Recommended Models for Cancellation Prediction:

Model	Rationale for Selection	Expected Performance Profile
Logistic Regression	Interpretable baseline model; works well with binary target and mix of continuous + categorical features (after encoding)	Accuracy ~80%; AUC ~0.82; good recall for non-cancellations
Random Forest	Handles mixed data types and non-linear interactions; robust to class imbalance; aligns with top-ranked features from Rank widget	Accuracy ~85–87%; AUC ~0.89; strong precision on cancellation class
Gradient Boosting / XGBoost	Best suited for structured tabular data with strong feature interactions (deposit_type × market_segment interactions)	Accuracy ~88%; AUC ~0.91; highest F1-score overall
Naive Bayes	Baseline probabilistic model; fast on categorical features	Accuracy ~74%; AUC ~0.76; useful as benchmark only

5.2 Critical Predictive Features (from Feature Ranking):

Based on the Rank widget output using Information Gain and Gini Decrease metrics, the following features are identified as the most important for cancellation prediction:

Rank	Feature	Type	Analytical Justification
1	deposit_type	Categorical	Non-Refund deposits show near-100% cancellation rates; single most discriminative feature
2	lead_time	Numerical	Higher lead time strongly associated with cancellation; confirmed via boxplot and correlation analysis
3	total_of_special_requests	Numerical	Negative correlation with cancellation reflects booking commitment level
4	previous_cancellations	Numerical	Repeat cancellers are highly predictive of future cancellations
5	market_segment	Categorical	Online TA vs. Direct bookings show different cancellation profiles
6	customer_type	Categorical	Transient-Party customers exhibit higher cancellation rates than Contract customers
7	required_car_parking_spaces	Numerical	Guests requesting parking are less likely to cancel (proxy for commitment)
8	booking_changes	Numerical	Multiple booking modifications correlate with higher commitment and lower cancellation

5.3 Key Findings & Business Recommendations:

5.3.1 Primary Insights:

- Cancellation rate: approximately 37% of all bookings are cancelled, representing significant potential revenue loss
- Non-Refund deposit bookings paradoxically cancel at the highest rate (~99%), suggesting a systematic data artifact or OTA booking behavior requiring policy review
- City hotels experience higher cancellation rates than Resort hotels, likely due to business traveler flexibility and OTA dominance in urban markets
- Lead time is the single most actionable predictor: bookings made 90+ days in advance should trigger automated re-confirmation workflows
- Portugal (PRT) dominates the guest origin distribution, suggesting targeted marketing investment would yield high returns

5.3.2 Operational Recommendations

- Implement an automated re-confirmation system for bookings with lead_time > 90 days
- Review Non-Refund deposit policy with OTA partners to investigate the anomalous 99% cancellation rate
- Introduce loyalty program enhancements for direct-booking channels (which show lower cancellation rates)
- Deploy the trained cancellation prediction model in the reservation system to flag high-risk bookings for proactive management
- Prioritize upselling services (breakfast, spa, parking) at booking time — each additional special request reduces cancellation probability

5.4 Conclusion:

The Hotel Analytics project successfully demonstrates the complete data mining lifecycle across three complementary tool ecosystems. The SQL database provides a robust operational foundation for hotel management, while the Python-based EDA and Orange visual workflow together deliver a rich, multi-perspective analysis of booking behavior.

The most impactful finding is that cancellation in hotel bookings is not random — it is strongly governed by a small set of behavioral and booking-context features, particularly deposit type, lead time, and prior cancellation history. This finding is consistent across all three analysis methodologies (SQL aggregation, Python statistical analysis, and Orange feature ranking), lending high confidence to the conclusions.

Future work should incorporate supervised machine learning classification models (tested in Orange's Test & Score widget with cross-validation) and a Power BI dashboard for executive-level visualization of the KPIs identified in this report.

5.4 Question and their Answers:

Q1:

Create a view vw_GuestMaster that returns: • GuestID, FullName, NationalID/Passport, Phone, Email, City • TotalStays (count of completed reservations) • LifetimeSpend (sum of paid amounts) Then query the Top 20 guests by LifetimeSpend

```sql
--Guest Master View
CREATE VIEW vw_GuestMaster AS
SELECT
    G.GuestID, G.FullName, G.NationalID, G.PhoneNumber, G.Email, G.City,
    COUNT(R.ReservationID) AS TotalStays,
    ISNULL(SUM(P.AmountPaid), 0) AS LifetimeSpend
FROM Guests G
LEFT JOIN Reservations R ON G.GuestID = R.GuestID AND R.ReservationStatus = 'Checked-out'
LEFT JOIN Invoices I ON R.ReservationID = I.ReservationID
LEFT JOIN Payments P ON I.InvoiceID = P.InvoiceID
GROUP BY G.GuestID, G.FullName, G.NationalID, G.PhoneNumber, G.Email, G.City;
GO

-- Query: Top 20 guests by LifetimeSpend
SELECT TOP 20 * FROM vw_GuestMaster ORDER BY LifetimeSpend DESC;
```

	GuestID	FullName	NationalID	PhoneNumber	Email	City	TotalStays	LifetimeSpend
1	39	Guest_Mixed_39	ID_4039	+139-555-000	customer39@global.com	New York	1	1740.00
2	139	Guest_Mixed_139	ID_4139	+239-555-000	customer139@global.com	New York	1	1740.00
3	87	Guest_Mixed_87	ID_4087	+187-555-000	customer87@global.com	New York	1	1580.00
4	42	Guest_Asian_42	ID_4042	+142-555-000	customer42@global.com	Tokyo	1	1575.00
5	142	Guest_Asian_142	ID_4142	+242-555-000	customer142@global.com	Tokyo	1	1575.00
6	89	Guest_Western_89	ID_4089	+189-555-000	customer89@global.com	Paris	1	1570.00
7	86	Guest_Asian_86	ID_4086	+186-555-000	customer86@global.com	Tokyo	1	1550.00
8	44	Guest_Arab_44	ID_4044	+144-555-000	customer44@global.com	Cairo	1	1545.00
9	144	Guest_Arab_144	ID_4144	+244-555-000	customer144@global.com	Cairo	1	1545.00
10	37	Guest_Western_37	ID_4037	+137-555-000	customer37@global.com	Paris	1	1540.00
11	137	Guest_Western_...	ID_4137	+237-555-000	customer137@global.com	Paris	1	1540.00
12	41	Guest_Western_41	ID_4041	+141-555-000	customer41@global.com	Paris	1	1530.00
13	88	Guest_Arab_88	ID_4088	+188-555-000	customer88@global.com	Cairo	1	1530.00
14	93	Guest_Western_93	ID_4093	+193-555-000	customer93@global.com	Paris	1	1530.00
15	141	Guest_Western_...	ID_4141	+241-555-000	customer141@global.com	Paris	1	1530.00
16	40	Guest_Arab_40	ID_4040	+140-555-000	customer40@global.com	Cairo	1	1525.00
17	94	Guest_Asian_94	ID_4094	+194-555-000	customer94@global.com	Tokyo	1	1525.00
18	140	Guest_Arab_140	ID_4140	+240-555-000	customer140@global.com	Cairo	1	1525.00
19	35	Guest_Mixed_35	ID_4035	+135-555-000	customer35@global.com	New York	1	1524.00
20	135	Guest_Mixed_135	ID_4135	+235-555-000	customer135@global.com	New York	1	1524.00

Q2:

Create a view vw_RoomAvailability that shows for each room:
RoomID, RoomNumber, RoomType, NightlyRate AvailabilityStatus for a given date range (Available / Booked) Then write a query to list all available rooms between two input dates

```sql
--Room Availability View
CREATE VIEW vw_RoomBookings AS
SELECT
    R.RoomNumber, RT.TypeName, RT.BasePrice, RES.CheckInDate, RES.CheckOutDate
FROM Rooms R
JOIN Room_Types RT ON R.RoomTypeID = RT.RoomTypeID
LEFT JOIN Reservations RES ON R.RoomNumber = RES.RoomNumber
WHERE RES.ReservationStatus IN ('Booked', 'Checked-in');
GO

-- Query: Find available rooms between '2026-03-05' and '2026-03-07'
SELECT R.RoomNumber, RT.TypeName, RT.BasePrice
FROM Rooms R
JOIN Room_Types RT ON R.RoomTypeID = RT.RoomTypeID
WHERE R.RoomNumber IN (

    SELECT RoomNumber FROM Rooms

    EXCEPT

    SELECT RoomNumber FROM vw_RoomBookings
    WHERE CheckInDate < '2026-03-07' AND CheckOutDate > '2026-03-05'
);
```

	RoomNumber	TypeName	BasePrice
1	101	Single	100.00
2	102	Single	100.00
3	103	Single	100.00
4	104	Single	100.00
5	111	Single	100.00
6	112	Single	100.00
7	113	Single	100.00
8	114	Single	100.00
9	121	Double	150.00
10	122	Double	150.00
11	123	Double	150.00
12	124	Double	150.00
13	131	Double	150.00
14	132	Double	150.00
15	133	Double	150.00
16	134	Double	150.00
17	141	Suite	300.00
18	142	Suite	300.00
19	143	Suite	300.00
20	144	Suite	300.00
21	151	Single	100.00
22	152	Double	150.00
23	153	Double	150.00
24	154	Suite	300.00
25	155	Family	250.00

Q3:

Write a query that calculates occupancy rate per day for a date range:

Date

TotalRooms

OccupiedRooms

OccupancyRate% = OccupiedRooms / TotalRooms * 100

```sql
--Daily Occupancy Rate
DECLARE @Today DATE = '2026-03-02';

DECLARE @TotalRooms FLOAT = (SELECT COUNT(*) FROM Rooms);

SELECT
    @Today AS [AnalysisDate],
    @TotalRooms AS [TotalCapacity],
    COUNT(*) AS [OccupiedRooms],
    (COUNT(*) / @TotalRooms) * 100 AS [OccupancyRate%]
FROM Reservations
WHERE @Today >= CheckInDate AND @Today < CheckOutDate
AND ReservationStatus IN ('Booked', 'Checked-in');
```

	AnalysisDate	TotalCapacity	OccupiedRooms	OccupancyRate%
1	2026-03-02	55	40	72.7272727272727

Q4:

Create a view vw_ReservationDetails that returns:
ReservationID, GuestName, RoomNumber, RoomType
CheckInDate, CheckOutDate, Nights
ReservationStatus (Booked/Checked-in/Checked-out/Cancelled)
TotalRoomCharge (Nights × NightlyRate)

```sql
--Reservation Details View

CREATE VIEW vw_ReservationDetails AS
SELECT
    R.ReservationID, G.FullName AS GuestName, R.RoomNumber, RT.TypeName AS RoomType,
    R.CheckInDate, R.CheckOutDate, R.Nights, R.ReservationStatus,
    (R.Nights * RT.BasePrice) AS TotalRoomCharge
FROM Reservations R
JOIN Guests G ON R.GuestID = G.GuestID
JOIN Rooms RM ON R.RoomNumber = RM.RoomNumber
JOIN Room_Types RT ON RM.RoomTypeID = RT.RoomTypeID;

select * from vw_ReservationDetails
```

	ReservationID	GuestName	RoomNumber	RoomType	CheckInDate	CheckOutDate	Nights	ReservationStatus	TotalRoomCharge
1	1	Guest_Western_1	102	Single	2026-03-09	2026-03-14	5	Checked-out	500.00
2	2	Guest_Asian_2	103	Single	2026-03-10	2026-03-15	5	Checked-out	500.00
3	3	Guest_Mixed_3	104	Single	2026-03-11	2026-03-16	5	Checked-out	500.00
4	4	Guest_Arab_4	105	Single	2026-03-12	2026-03-17	5	Checked-out	500.00
5	5	Guest_Western_5	106	Single	2026-03-13	2026-03-18	5	Checked-out	500.00
6	6	Guest_Asian_6	107	Single	2026-03-14	2026-03-19	5	Checked-out	500.00
7	7	Guest_Mixed_7	108	Single	2026-03-15	2026-03-20	5	Checked-out	500.00
8	8	Guest_Arab_8	109	Single	2026-03-16	2026-03-21	5	Checked-out	500.00
9	9	Guest_Western_9	110	Single	2026-03-17	2026-03-22	5	Checked-out	500.00
10	10	Guest_Asian_10	111	Single	2026-03-18	2026-03-23	5	Checked-out	500.00
11	11	Guest_Mixed_11	112	Single	2026-03-19	2026-03-24	5	Checked-out	500.00
12	12	Guest_Arab_12	113	Single	2026-03-20	2026-03-25	5	Checked-out	500.00
13	13	Guest_Western...	114	Single	2026-03-21	2026-03-26	5	Checked-out	500.00
14	14	Guest_Asian_14	115	Single	2026-03-22	2026-03-27	5	Checked-out	500.00
15	15	Guest_Mixed_15	116	Double	2026-03-23	2026-03-28	5	Checked-out	750.00

Q5:

Write a query showing cancellation performance by month:
YearMonth
TotalReservations
CancelledReservations
CancellationRate% Also list the Top 5 cancellation reasons (if stored).

```sql
--Cancellation Analysis
SELECT
    FORMAT(CheckInDate, 'yyyy-MM') AS [Month],
    COUNT(*) AS [Total Bookings],
    SUM(IIF(ReservationStatus = 'Cancelled', 1, 0)) AS [Cancelled Count],
    SUM(IIF(ReservationStatus = 'Cancelled', 1.0, 0)) * 100 / COUNT(*) AS [Cancellation Rate %]

FROM Reservations
GROUP BY FORMAT(CheckInDate, 'yyyy-MM');

-- Top 5 Reasons
SELECT TOP 5 CancellationReason, COUNT(*) as Count_
FROM Reservations
WHERE ReservationStatus = 'Cancelled'
GROUP BY CancellationReason
ORDER BY Count_ DESC;
```

	Month	Total Bookings	Cancelled Count	Cancellation Rate %
1	2026-01	1	0	0.000000
2	2026-03	363	86	23.691460
3	2026-04	241	65	26.970954

	CancellationReason	Count_
1	Budget Issues	50
2	Personal Emergency	50
3	Flight Delay	50
4	Change of Plans	1

Q6:

Create a view vw_ServiceRevenue that returns:
ServiceName, YearMonth
TotalUsageCount
TotalServiceRevenue Then query Top 5 services by revenue for the last 3 months

```sql
--Services Revenue View
CREATE VIEW vw_ServiceRevenue AS
SELECT
    S.ServiceName,
    FORMAT(SU.RequestTime, 'yyyy-MM') AS YearMonth,
    COUNT(SU.UsageID) AS TotalUsageCount,
    SUM(SU.Quantity * S.ServicePrice) AS TotalServiceRevenue
FROM Services_ S
JOIN Service_Usage SU ON S.ServiceID = SU.ServiceID
GROUP BY S.ServiceName, FORMAT(SU.RequestTime, 'yyyy-MM');
GO

-- Query: Top 5 services last 3 months
SELECT TOP 5 ServiceName, SUM(TotalServiceRevenue) as Revenue
FROM vw_ServiceRevenue
WHERE YearMonth >= FORMAT(DATEADD(MONTH, -3, GETDATE()), 'yyyy-MM')
GROUP BY ServiceName ORDER BY Revenue DESC;
```

	ServiceName	Revenue
1	Safari Tour	1920.00
2	Oriental Spa	750.00
3	Airport Pickup	640.00
4	Extra Bed	560.00
5	Mini Bar Restock	450.00

Q7:

Create a view vw_InvoiceAging that shows:
InvoiceID, GuestName, InvoiceDate, TotalAmount, PaidAmount, OutstandingAmount
AgingBucket: 0–7, 8–30, 31–60, 60+ days Then query all outstanding invoices in the 60+ bucket

```sql
--Invoice Aging & Outstanding Balances
CREATE VIEW vw_InvoiceAging AS
SELECT
    I.InvoiceID, G.FullName AS GuestName, I.InvoiceDate, I.TotalAmount,
    ISNULL(SUM(P.AmountPaid), 0) AS PaidAmount,
    (I.TotalAmount - ISNULL(SUM(P.AmountPaid), 0)) AS OutstandingAmount,
    DATEDIFF(DAY, I.InvoiceDate, GETDATE()) AS DaysOld,
    CASE
        WHEN DATEDIFF(DAY, I.InvoiceDate, GETDATE()) <= 7 THEN '0–7'
        WHEN DATEDIFF(DAY, I.InvoiceDate, GETDATE()) <= 30 THEN '8–30'
        WHEN DATEDIFF(DAY, I.InvoiceDate, GETDATE()) <= 60 THEN '31–60'
        ELSE '60+'
    END AS AgingBucket
FROM Invoices I
JOIN Reservations R ON I.ReservationID = R.ReservationID
JOIN Guests G ON R.GuestID = G.GuestID
LEFT JOIN Payments P ON I.InvoiceID = P.InvoiceID
GROUP BY I.InvoiceID, G.FullName, I.InvoiceDate, I.TotalAmount;

--all outstanding invoices in the 60+ bucket
SELECT * FROM vw_InvoiceAging
WHERE AgingBucket = '60+'
AND OutstandingAmount > 0
ORDER BY OutstandingAmount DESC;
```

Q8:

Write a query that returns (monthly):
YearMonth
TotalPaid
PaidByCash, PaidByCard, PaidOnline, PaidByCompany (if applicable)
Failed/Refunded counts (if stored)

```sql
--Payment Method Breakdown
SELECT
    FORMAT(PaymentDate, 'yyyy-MM') AS YearMonth,
    SUM(AmountPaid) AS TotalPaid,
    SUM(CASE WHEN PaymentMethod = 'Cash' THEN AmountPaid ELSE 0 END) AS PaidByCash,
    SUM(CASE WHEN PaymentMethod = 'Credit Card' THEN AmountPaid ELSE 0 END) AS PaidByCard,
    SUM(CASE WHEN PaymentMethod = 'Online' THEN AmountPaid ELSE 0 END) AS PaidByOnline
FROM Payments
WHERE PaymentStatus = 'Success'
GROUP BY FORMAT(PaymentDate, 'yyyy-MM');
```

	YearMonth	TotalPaid	PaidByCash	PaidByCard	PaidByOnline
1	2026-03	139743.00	68205.00	71538.00	0.00

Q9:

Assume Housekeeping has cleaning logs per room per date. Write a query to compute:
CleaningCount per room (last 30 days)
Average time to clean (EndTime − StartTime) if available
Rooms cleaned late (after a defined cutoff time) *(Bonus: create view vw_HousekeepingPerformance.)*

```sql
-- 10. Manual Housekeeping Log Entry (Room cleaning record)
-- StaffID 11-25 are Housekeeping staff
INSERT INTO Housekeeping_Logs (RoomNumber, StaffID, LogDate, StartTime, EndTime)
VALUES ('101', 11, CAST(GETDATE() AS DATE), '10:00:00', '11:00:00'),
       ('155', 12, CAST(GETDATE() AS DATE), '12:00:00', '13:00:00');

       GO
```

```sql
--Housekeeping & Room Turnover Report

CREATE VIEW vw_HousekeepingPerformance AS
SELECT
    RoomNumber,
    COUNT(LogID) AS CleaningCount,
    AVG(DATEDIFF(MINUTE, StartTime, EndTime)) AS AvgCleanTimeMinutes,
    SUM(CASE WHEN CAST(EndTime AS TIME) > '15:00:00' THEN 1 ELSE 0 END) AS CleanedLateCount
FROM Housekeeping_Logs
WHERE LogDate >= DATEADD(DAY, -30, GETDATE())
GROUP BY RoomNumber;

SELECT * FROM vw_HousekeepingPerformance
GO

--Staff Performance View


CREATE VIEW vw_StaffPerformance AS
SELECT
    S.FullName AS StaffName, S.Role,
    COUNT(DISTINCT R.ReservationID) AS ReservationsHandled,
    SUM(CASE WHEN R.ReservationStatus = 'Checked-in' THEN 1 ELSE 0 END) AS CheckInsProcessed,
    SUM(CASE WHEN R.ReservationStatus = 'Checked-out' THEN 1 ELSE 0 END) AS CheckOutsProcessed,
    COUNT(DISTINCT P.PaymentID) AS TotalPaymentsProcessed,
    ISNULL(SUM(P.AmountPaid), 0) AS TotalRevenueProcessed
FROM Staff S
LEFT JOIN Reservations R ON S.StaffID = R.StaffID
LEFT JOIN Invoices I ON R.ReservationID = I.ReservationID
LEFT JOIN Payments P ON I.InvoiceID = P.InvoiceID
WHERE R.CheckInDate >= DATEADD(DAY, -30, GETDATE()) OR R.CheckInDate IS NULL
GROUP BY S.FullName, S.Role;
GO
```

Q10:

Create a view vw_Staff Performance for last 30 days:

StaffName, Role

ReservationsHandled (count)

CheckInsProcessed, CheckOutsProcessed

TotalPaymentsProcessed

TotalRevenueProcessed Then query the Top 10 staff by TotalRevenueProcessed

```sql
CREATE VIEW vw_StaffPerformance AS
SELECT
    S.FullName AS StaffName, S.Role,
    COUNT(DISTINCT R.ReservationID) AS ReservationsHandled,
    SUM(CASE WHEN R.ReservationStatus = 'Checked-in' THEN 1 ELSE 0 END) AS CheckInsProcessed,
    SUM(CASE WHEN R.ReservationStatus = 'Checked-out' THEN 1 ELSE 0 END) AS CheckOutsProcessed,
    COUNT(DISTINCT P.PaymentID) AS TotalPaymentsProcessed,
    ISNULL(SUM(P.AmountPaid), 0) AS TotalRevenueProcessed
FROM Staff S
LEFT JOIN Reservations R ON S.StaffID = R.StaffID
LEFT JOIN Invoices I ON R.ReservationID = I.ReservationID
LEFT JOIN Payments P ON I.InvoiceID = P.InvoiceID
WHERE R.CheckInDate >= DATEADD(DAY, -30, GETDATE()) OR R.CheckInDate IS NULL
GROUP BY S.FullName, S.Role;
GO
```